

MeTTa Formal Specification: Draft

Ben Goertzel

September 8, 2025

Abstract

This document provides a formal specification of the MeTTa programming language. The aim is to define a simple but precise grammar for MeTTa, together with a compact small-step operational semantics that matches actual practice and supports transpilation into MeTTa-IL, a graph-structured lambda-theoretic intermediate language. By capturing both the syntactic surface and the runtime dynamics, the specification aims to enable independent implementations, formal reasoning about programs, and correct-by-construction compilation pipelines. The presentation is self-contained and illustrated with detailed worked examples including recursive functions, Python interoperation, algebraic data types, and a chess queens solver, all translated into MeTTa-IL. Our goal is to furnish the MeTTa community with a standard reference that plays a role similar to the JVM specification or the Ethereum Yellow Paper, ensuring semantic clarity and portability across implementations.

This document is a first effort and may have omissions or errors and is in need of some checking and perhaps correction by the MeTTa development community.

Contents

1	Introduction	6
1.1	Scope and Motivation	7
1.2	Core Conformance Set	9
1.3	Choosing between MeTTa, MeTTa-IL, MM2/MORK and PyMeTTa	9
2	MeTTa Syntax Specification	10
2.1	Scope and Overview	10
2.2	Lexical Structure	11
2.2.1	Whitespace and Comments	11
2.2.2	Basic Tokens	11
2.2.3	Identifiers	11
2.3	Program Structure	12
2.3.1	Core Syntactic Categories	12
2.3.2	Patterns	12

2.3.3	Top-Level Forms	12
2.4	Type System	13
2.4.1	Type Expressions	13
2.4.2	Static Semantics (Minimal Typing Fragment)	13
2.5	Evaluation Model and Special Forms	14
2.5.1	Designated Forms with Non-Strict Arguments	14
2.5.2	Boolean Short-Circuit Forms	14
2.6	Standard Library Forms	15
2.6.1	Space Operations	15
2.6.2	Python Interoperability	15
2.6.3	Grounded Numeric and Relational Operators	15
2.6.4	Console I/O	15
2.6.5	Assertions	15
2.7	Advanced Features	16
2.7.1	Sealed Variables (Alpha-Conversion)	16
2.7.2	Syntactic Sugar (Optional)	16
2.7.3	Library definitions for <code>if</code> and <code>quote</code> (H-E MeTTa)	16
2.8	Complete Grammar (EBNF)	16
2.9	Implementation Guidance	19
2.9.1	Parsing Pipeline	19
2.9.2	Transpiler Requirements	19
2.9.3	Why This Grammar Suffices for MeTTa-IL	19
2.9.4	Minimal examples that parse under this grammar	20
3	Compact Small-Step Operational Semantics for MeTTa	21
3.1	Overview and Approach	21
3.2	Abstract Syntax	21
3.3	Machine State and Configuration	22
3.3.1	Freshness Service	22
3.3.2	Evaluation Conventions	22
3.4	Core Reduction Rules	22
3.4.1	Nondeterminism and Result Ordering	23
3.5	Grounded Operations	23
3.5.1	Basic Arithmetic and Logic	24
3.5.2	Numeric and Relational Operators	24
3.5.3	Console I/O	24
3.6	Surface Construct Semantics	24
3.6.1	Equality and Function Clauses	24
3.6.2	Single-Sided Matching (Optional)	24
3.6.3	Control Forms	24
3.6.4	Nondeterminism Operators	25
3.6.5	Expression Structure Operations	25
3.6.6	Space API	25
3.6.7	Tokens and Modules	26
3.6.8	Sealed (Alpha-Conversion)	26
3.7	Python Interoperability	26

3.7.1	Interop Lifecycle	27
3.8	Error Handling	27
3.8.1	Assertions	27
3.8.2	Error Model and Propagation	27
3.9	Process Interpretation (Concurrency)	28
3.10	Static Typing (Optional)	28
3.11	Transpilation to MeTTa-IL	28
3.12	Worked micro-examples (sanity checks)	28
3.13	Design Decisions and Non-Assumptions	29
3.14	Summary for Transpiler Implementers	29
4	Alternative Presentation: MeTTa Operational Semantics via the MeTTa Calculus	30
4.1	MeTTa Calculus fragments used here	30
4.2	Distinguished spaces and an “emit” macro	30
4.3	Compilation of program items to processes	30
4.4	Reading the machine rules through COMM	31
4.5	Special forms and library heads	31
4.6	Spaces, modules, and tokens	32
4.7	Sealed (alpha-conversion)	32
4.8	Grounded numerics, interop, and I/O	32
4.9	Errors and assertions	32
4.10	Concurrency and transactional reading	33
4.11	Correspondence (bisimulation up to scheduling)	33
5	MeTTa-IL: A Concrete Intermediate Language Schema	33
5.1	Motivation and Scope	33
5.2	BNFC Textual IL Conventions	34
5.3	Design Goals	35
5.4	Venus front-end pipeline (Modules to Graph IR)	35
5.5	Textual Program Format (BNFC-compatible)	38
5.6	Core Structure (Abstract)	39
5.6.1	Sorts and Base Types	39
5.7	Term Constructors and Strictness	39
5.7.1	Core Data and Application	39
5.7.2	Equations and Rules	39
5.7.3	Control Forms	39
5.7.4	Nondeterminism	40
5.7.5	Spaces and Queries	40
5.7.6	Sealing and Interop	40
5.7.7	Assertions and Diagnostics	41
5.7.8	Patterns	41
5.8	Operational Semantics (Edge Families)	41
5.8.1	Core Rewriting	41
5.8.2	Spaces	41
5.8.3	Control Flow	42

5.8.4	Nondeterminism	42
5.8.5	Sealed, Output	42
5.8.6	Python Interop	42
5.8.7	Assertions and Errors	42
5.9	Lowering from Surface to IL	42
5.9.1	Pattern Translation	43
5.9.2	Control Forms	43
5.9.3	Other Forms	43
5.9.4	Grounded Operators	43
5.10	Strictness Table	43
5.11	Static Well-Formedness	44
5.12	Conformance	44
5.13	Worked Micro-Examples (Textual IL)	44
5.14	From MeTTa Calculus to MeTTa-IL (one-step view)	45
6	User-Customizable Evaluation Order in MeTTa	46
6.1	Overview and Motivation	46
6.2	Surface Syntax	47
6.2.1	Evaluation Policy Declarations	47
6.2.2	Evaluation Modes	47
6.2.3	Examples	48
6.3	Operational Semantics	48
6.3.1	Extended Machine State	48
6.3.2	Policy Structure	48
6.3.3	Argument Preparation Phase	48
6.3.4	Thunk Semantics (By-Need)	49
6.3.5	Result Policy	49
6.3.6	Policy Precedence and Special Cases	49
6.4	Design Principles	50
6.5	IL Extension (BNFC textual IL)	50
6.5.1	Top-level policy declarations	50
6.5.2	New heads (textual IL)	51
6.5.3	Edge families (small-step)	51
6.5.4	Application elaboration (textual IL)	51
6.5.5	Lowering from surface to textual IL	52
6.5.6	Well-formedness	52
6.6	Examples (BNFC textual IL)	52
6.6.1	Short-circuit boolean with on-demand forcing	52
6.6.2	By-need memoization	53
6.6.3	Parallel evaluation of a strict argument	53
6.7	Summary	53

7	A Few Concrete Examples	54
7.1	Syntax and Semantics of Simple MeTTa Code Examples	54
7.1.1	EXAMPLE A — recursive mapper over expressions	54
7.1.2	EXAMPLE B — Python interop, grounded atoms, key-word args	57
7.1.3	Why these explanations/ILs line up with the formal semantics	59
7.2	Example: The Red and Black Lambda Calculus in MeTTa	59
7.2.1	Code Snippets in Question	60
7.2.2	What each fragment is syntactically	61
7.2.3	How the semantics justifies what happens	63
7.2.4	A compact MeTTa-IL that a transpiler can emit	64
7.2.5	Why this IL is faithful and useful to your transpiler	66
7.2.6	Optional: how a loader would use these results	67
7.3	Example: Chess Queens Problem in MeTTa	68
7.3.1	The snippet (for reference)	68
7.3.2	How each form fits the surface grammar	69
7.3.3	Intended data representation	71
7.3.4	Operational semantics walk-through	71
7.3.5	Translation to MeTTa-IL (the transpiler’s core IR)	74
7.3.6	Why the snippet behaves correctly under the formal semantics	76
7.3.7	A tiny reduction trace (sketch)	77
7.4	Concrete MeTTa example using sealed	78
7.5	Worked example: sequential composition with progn	82
7.6	Concrete Examples of User-Configurable Evaluation Order	85
7.6.1	Example 1 — Left-to-right vs right-to-left evaluation	85
7.6.2	Example 2 — Short-circuiting with by-need (lazy) argument	88
7.6.3	Synthesis: evaluation order as a first-class concern	90
7.6.4	Takeaway	91

Note on Authorship

While this document was put together by Ben Goertzel, and it is Ben (and/or the LLMs he used to help, or the demons who may or may not have possessed his brain at any given point in our collectively-hallucinated time axis!) to whom any errors here should be attributed ... the actual programming languages documented here are the work of a group of people – most notably Alexey Potapov and Vitaly Bogdanov who conceived the original MeTTa concept collaboratively with me, and then invented and led implementation of the first version of MeTTa (Hyperon Experimental), and then Greg Meredith and Mike Stay who invented and implemented MeTTa-IL (and Greg also suggested a number of important MeTTa language features) ... Adam Vander-vorst, Luke Peterson and Jonathan Warrell for help refining and implementing the MeTTa concept and details in various ways ... (and apologies to any of my super-important colleagues whom I have omitted to mention here while typing late at night on insufficient sleep!) ... and a whole bunch of AI developers in the Hyperon and

SingularityNET and F1R3FLY orbit who have been coding in MeTTa and related languages, or in some cases building infrastructural tools for MeTTa, thus allowing us to make progress on understanding what MeTTa is supposed to be.

1 Introduction

MeTTa ¹ is designed as a foundational language for programming intelligent systems, developed in the context of the Hyperon and SingularityNET projects [2]. Its surface syntax is minimal, consisting of fully parenthesized S-expressions augmented with a small set of special forms. The semantics of the language are defined by equations, unification, and rewriting, in a style reminiscent of logic programming but extended with higher-order functions, nondeterminism, and reflection. MeTTa is intended to be simultaneously simple for human programmers and expressive enough to serve as a substrate for advanced AGI applications.

This document presents a formal specification of MeTTa. In particular, we provide:

- A complete surface grammar expressed in EBNF, capturing the lexical structure, patterns, type expressions, and designated special forms.
- A compact small-step operational semantics based on a four-register abstract machine (input, knowledge base, workspace, output), adapted from the Meta-MeTTa specification [1] (and inspired also in some places by the MeTTa calculus [3])
- A mapping of these rules to MeTTa-IL [4], a graph-structured lambda-theoretic intermediate representation that enables reasoning about correctness and supports efficient compilation to multiple execution backends.
- Worked examples demonstrating the syntax, semantics, and IL translation for a variety of MeTTa programs.

By spelling out both syntax and semantics in this way, we make possible independent, compliant implementations of MeTTa, as well as meta-theoretic analysis of programs and compilation schemes. The specification is intended to support correct-by-construction development, reduce ambiguity in program meaning, and provide a stable foundation for future extensions of the language.

In constructing this grammar I have relied mainly on the working MeTTa code available online in the main HE-MeTTa repo on TrueAGI GitHub and a couple other GitHub repos. I.e., this is an attempt to capture what "HE-MeTTa" (Hyperon Experimental MeTTa) is in practice right now, with just a few additions and tweaks based on what the developer community would like it to be. Two features under discussion but not yet implemented (single-sided

¹Originally a pseudo-acronym for "Meta Type Talk", though I've come to prefer "Meta Type Theory Assembler" which ChatGPT hallucinated for me one day – take your pick or make up something new!

pattern matching, and **progn** for sequential composition) are included, and a possible new feature (user-configurable evaluation order) is tentatively proposed for inclusion.

1.1 Scope and Motivation

The goal of this document is to make MeTTa easy to implement correctly and to reason about. Concretely, we provide (i) a complete and minimal surface grammar that matches real programs, (ii) a compact small-step operational semantics that explains evaluation in terms of equations, unification, and rewriting, and (iii) a precise mapping to an intermediate language (MeTTa-IL) that supports portable compilation, testing, and formal analysis. The specification aims to reduce ambiguity across implementations, enable correct-by-construction transpilers and interpreters, and serve as a stable foundation for tooling and future extensions.

In scope. This specification covers the following, normatively:

- **Concrete syntax (surface grammar).** Lexical structure, identifiers, variables, tokens, atoms; S-expression forms for facts, type declarations (`: head type`), and definitional equalities (`= lhs rhs`); patterns (including structured patterns) and type expressions.
- **Designated control forms and their strictness.** `let`, `let*`, `case`, `match`, `superpose`, `collapse`. For each, the non-strict (Atom-typed) argument positions are specified and preserved by the semantics.
- **Core operational semantics.** A small-step abstract machine with input, knowledge base, workspace, and output registers; the *Query*, *Chain*, *Transform*, *Add/Remove*, and *Output* rules; evaluation of grounded operations (booleans, arithmetic, basic data constructors).
- **Spaces and bindings.** Grounded operations to create and manipulate spaces; the effect of `bind!` (symbol-to-space binding) and `import!` (module load + optional symbol installation). No separate token-expansion phase. phase separation between token expansion (parse-time) and evaluation.
- **Nondeterminism.** Production of multiple results via multiple equations and `superpose`, and materialization via `collapse`.
- **Interop primitives.** The existence and calling discipline of `py-atom`, `py-dot`, and `Kwargs` as grounded operations with strictness of evaluated arguments documented.
- **sealed.** A grounded primitive for on-demand alpha-conversion: `(sealed (v_1 ... v_k) t)` freshens all variable occurrences in `t` except the listed ones; this is specified by a single reduction rule (fresh-name generation).

- Sequential composition via **progn**, which evaluates a block of forms left-to-right and returns the last result, is included as a core special form.
- **MeTTa-IL mapping**. A concrete schema for IL terms, equations, and scripts; a deterministic translation from surface constructs to IL; and the correspondence between IL execution and the abstract machine rules.

Out of scope. The following are deliberately not fixed by this specification:

- Algorithmic complexity, cost semantics, and performance guarantees.
- Concrete memory model, garbage collection, concurrency scheduling, and parallel execution strategies.
- Complete standard library catalog beyond the designated special forms and the named grounded primitives above.
- Host-language specifics of foreign functions (e.g., Python object lifetimes, numeric tower exactness) beyond the call discipline and argument evaluation order.
- Security, capability, and sandboxing policies.
- Proofs of type soundness or confluence; we provide the needed hooks (types and rules) but do not prove meta-theorems

Extensions of Current HE-MeTTa. One feature is specified as optional here, with well-defined integration points:

- **Evaluation policies.** Per-symbol **eval-policy** declarations to choose *by-value*, *by-name*, or *by-need* per argument, plus evaluation order and optional parallel hints. The core semantics are parameterized by a policy table; a thunking/forcing rule is provided for by-need.

Another couple simpler features are included as part of the language although they are not implemented in the default (HE) version of MeTTa, because their implementation has been recently requested by the developer community and they seem quite reasonable:

- **Single-sided matching.** An annotation that restricts rule application to a functional (callee-driven) direction. This does not affect the base machine; it constrains which *Query/Chain* matches are eligible.
- **progn** for convenient sequential composition

1.2 Core Conformance Set

Based on the particulars and perspective given here, we propose that engine claiming *MeTTa Core Conformance* MUST implement at minimum:

- Rewriting by equalities and unification (Query/Chain), facts.
- Special forms: `if`, `let`, `let*`, `case`, `match`, `quote`, `superpose`, `collapse`, `progn`, and `empty`.
- Expression structure ops: `car-atom`, `cdr-atom`, `cons-atom`.
- Spaces API: `new-space`, `add-atom`, `remove-atom`, `get-atoms`.
- Tokens/modules: `bind!`, `import!` with cycle behavior (Sec. 3.6.7).
- Grounded numerics/booleans/comparators: `+` `-` `*` `/`, `<` `<=` `>` `>=`, `==`, and boolean constants `True/False`.
- Console I/O grounders: `println!`, `trace!` (Sec. 3.5.3).

1.3 Choosing between MeTTa, MeTTa-IL, MM2/MORK and PyMeTTa

We are building a complex new programming language stack for AGI here, so it may be worth clarifying what I see as the unique value of each of the layers and tools involved. It should be understood that different developers may have different perspectives on these relative value and ideal use of each tool involved, so the comments here represent one (highly educated but not omniscient) perspective.

From a practical programmer's view, each layer in the stack serves a different purpose; as I see it:

MeTTa (surface language). The surface syntax is concise and human-friendly. It offers Lisp-like S-expressions and special forms (`if`, `let`, `match`, `progn`, `superpose`, etc.) that make rules, control flow, and nondeterminism easy to express. Programmers will normally use MeTTa for knowledge-base development, everyday AGI algorithms, and portable prototyping.

MeTTa-IL (intermediate language). The IL is a canonical, desugared form with explicit strictness, non-strict positions, and small-step rules for every construct. It is well-suited for tooling (static analyzers, transpilers, verifiers) and for debugging subtle evaluation issues. IL is also the basis for compilation to external backends. Tool and engine builders, or programmers who need a fully explicit view of execution, will choose IL.

MM2 / MORK (low-level graph operations). At the lowest level, the hypergraph engine (MORK) and MM2 expose the primitive operations on atoms, pattern matching, and indexing. This is the place to work when performance is critical, or when implementing extensions to the engine itself. Direct use of MM2/MORK is appropriate for building high-performance modules, custom indexing, or integration with other graph stores.

PyMeTTa and host-language integrations. Beyond the core layers, many programmers will prefer to embed MeTTa inside a host language such as Python, using PyMeTTa or similar bindings. These integrations let developers script experiments, drive MeTTa queries from ordinary Python code, and interoperate with the extensive scientific and machine learning ecosystem. PyMeTTa does not replace the surface language or IL; rather, it exposes them through a familiar runtime and object model, making it easier to orchestrate experiments, connect to external libraries, and integrate MeTTa programs into larger software projects. Programmers reach for PyMeTTa when they want convenience and ecosystem leverage, but still rely on MeTTa and MeTTa-IL as the normative language layers.

Summary. Programmers reach for MeTTa by default; when precision and portability matter they inspect or emit MeTTa-IL; when putting together applications needing limited and well-defined MeTTa functionalities they may use PyMeTTa or similar; when absolute control and performance are required they work directly with MM2/MORK.

2 MeTTa Syntax Specification

This section provides a complete surface grammar for MeTTa that is minimal, regular, and suitable for transpilation to MeTTa-IL. It captures the concrete syntax, abstract categories, and evaluation-order annotations needed for correct implementation.

2.1 Scope and Overview

This grammar covers:

- **Lexical structure:** Characters to tokens to S-expressions
- **Program structure:** Facts, type declarations (:), definitional equalities (=), meta-forms (bind!, import!), and evaluation directives !
- **Patterns and types:** Pattern matching syntax and type expressions
- **Evaluation model:** Special forms with non-strict argument positions
- **Standard library:** Space operations, Python interop, assertions

- **Extensions:** Optional single-sided matching ($\Rightarrow$) and sealed variables

Out of scope: Reduction rules, cost semantics, and concurrency belong to the operational semantics (Section 3), not the surface grammar.

2.2 Lexical Structure

2.2.1 Whitespace and Comments

```
Whitespace ::= U+0009 | U+000A | U+000D | U+0020
              | <other Unicode spaces>
Comment    ::= ';' <any-char-except-linebreak>*
```

Comments run from ; to end of line. Whitespace and comments may appear between any tokens.

2.2.2 Basic Tokens

```
(* Parentheses - MeTTa is fully parenthesized *)
LP ::= '('
RP ::= ')'

(* Literals *)
String ::= '"' { '\\"' | '\\n' | '\\t' | '\\\\'
              | <any-char-except["\\"]> } '"'
Number ::= ['+' | '-'] Digit+ ['. ' Digit+]
Bool    ::= 'True' | 'False'
Unit    ::= '(' ' ' ')'
```

2.2.3 Identifiers

```
(* Symbols ? any unquoted identifier *)
Symbol ::= IdStart IdCont*
IdStart ::= Letter | '_' | '+' | '-' | '*' | '/' |
              '<' | '>' | '=' | '%' | '!' | '?' | ':' |
              ',' | '@' | '^' |
              '~' | '|' | UnicodeLetter
IdCont  ::= IdStart | Digit

(* Variables ? for patterns and types *)
Variable ::= '$' Symbol

Letter      ::= 'A'..'Z' | 'a'..'z'
UnicodeLetter ::= <any Unicode code point with
                  XID_Start=true, excluding Letter>
Digit      ::= '0'..'9'
```

2.3 Program Structure

2.3.1 Core Syntactic Categories

```
Program ::= Toplevel*
Toplevel ::= Form | EvalItem
EvalItem ::= '!' Form      (* evaluation directive *)

Form      ::= Atom | List
Atom      ::= Variable | Symbol | Grounded | Unit
Grounded  ::= Number | String | Bool
List      ::= '(' Element* ')'
Element   ::= Atom | List
```

Any Form is legal at top level, but certain heads trigger special interpretation.

2.3.2 Patterns

Patterns are a syntactic subset of Form used in:

- Left-hand sides of (= pattern form)
- Match expressions (match space pattern result)
- Let bindings (let pattern value body)
- Case branches (case expr ((p1 r1) ...))

```
Pattern ::= PatAtom | PatList
PatAtom ::= Variable | Symbol | Number | String
          | Bool | Unit
PatList ::= '(' Pattern* ')'
```

Variables capture values; all other elements match literally.

2.3.3 Top-Level Forms

These are ordinary lists distinguished by their head symbol:

```
(* Type declaration *)
TypeDecl ::= '(' ':' Pattern TypeExpr ')'
```

```
(* Definitional equality *)
DefEq ::= '(' '=' Pattern Form ')'
```

```
(* Optional: Single-sided (forward-only) matching *)
DefEqOneSide ::= '(' '=>' Pattern Form ')'
```

```
(* Symbol binding and module import *)
```

```
SymbolBind ::= '(' 'bind!' Symbol Form ')'
```

```
Import      ::= '(' 'import!' Symbol Symbol ')'
```

- `(: f T)`: Declares type `T` for pattern `f`
- `(= lhs rhs)`: Bidirectional rewrite rule
- `(=> lhs rhs)`: Forward-only rewrite (optional extension)
- `(bind! name expr)`: Binds symbol `name` to the evaluated `expr`.
- `(import! name module)`: Imports `module` into a fresh space bound to symbol `name`.

2.4 Type System

2.4.1 Type Expressions

Types are first-class S-expressions:

```

TypeExpr ::= TypeAtom
           | '(' '-'> TypeExpr+ ')' (* function
                                     type *)
           | '(' TypeExpr TypeExpr* ')' (*
                                     application *)

TypeAtom ::= 'Type' | 'Atom' | 'Expression' | 'Symbol'
           | 'Bool' | 'Number' | 'String' | 'Nat'
           | 'hyperon::space::DynSpace'
           | Symbol (* user-defined *)
           | Variable (* type variable $t *)

```

Examples:

- (`-> Number Number Number`): Binary numeric function
- (`-> Bool Atom Atom $t`): Polymorphic conditional
- (`List Number`): Parameterized type

2.4.2 Static Semantics (Minimal Typing Fragment)

The type system is optional but provides useful validation. We write $\Gamma \vdash e : T$ for term typing.

Type checking rules:

- **Declaration lookup:** If $(\vdash f : T)$ is declared, then $\Gamma \vdash f : T$
- **Application:** If $\Gamma \vdash f : (- \multimap T_1 \dots T_n T)$ and $\Gamma \vdash e_i : T_i$ for each i , then $\Gamma \vdash (f\ e_1 \dots e_n) : T$
- **Equality:** Both sides must have the same type

Principal types for designated forms:

```
let   : (-> Pattern Form Atom $t)
case  : (-> Form (List (Pair Pattern Atom)) $t)
match : (-> Space Atom Atom $t)
```

*Note: In H-E MeTTa, `if` and `quote` are ordinary library functions with *Atom*-typed parameters.*

Type variables like `$t` are schematic and unify with any type.

2.5 Evaluation Model and Special Forms

2.5.1 Designated Forms with Non-Strict Arguments

Certain positions are marked *Atom*-typed and not evaluated before being passed to the form: ²

Form	Shape	Non-strict positions
let	(let pat val body)	1, 3
let*	(let* ((p1 v1) ...) body)	patterns, body
case	(case e ((p1 r1) ...))	branch results
match	(match space pat result)	2, 3
superpose	(superpose (a1 ...))	elements
collapse	(collapse atom)	1
sealed	(sealed (vars) body)	1, 2
progn	(progn form1 ...)	none (all strict)

2.5.2 Boolean Short-Circuit Forms

Standard boolean operators with short-circuit evaluation:

```
(: and (-> Bool Atom Bool))
(= (and True $q) $q)
(= (and False $q) False)

(: or (-> Bool Atom Bool))
(= (or True $q) True)
(= (or False $q) $q)

(: not (-> Bool Bool))
(= (not True) False)
(= (not False) True)
```

The second argument is *Atom*-typed to ensure lazy evaluation.

²In H-E MeTTa, `'if'` and `'quote'` are ordinary library functions, not language-level designated forms. Their laziness arises from *Atom*-typed parameters.

2.6 Standard Library Forms

2.6.1 Space Operations

<code>(new-space)</code>	<code>(* creates fresh space</code>
<code>*)</code>	
<code>(add-atom <Space> <Atom>)</code>	<code>(* adds atom as-is *)</code>
<code>(remove-atom <Space> <Atom>)</code>	<code>(* removes if present</code>
<code>*)</code>	
<code>(add-reduct <Space> <Form>)</code>	<code>(* adds reduced form *)</code>
<code>(get-atoms <Space> <Pattern>)</code>	<code>(* returns match tuple</code>
<code>*)</code>	

2.6.2 Python Interoperability

<code>(py-atom <String> [<Type>])</code>	<code>(* import Python object</code>
<code>*)</code>	
<code>(py-dot <Grounded> <Symbol>)</code>	<code>(* attribute access *)</code>
<code>(Kwargs (<key> <val>) ...)</code>	<code>(* keyword arguments *)</code>
 <code>(* Python collections *)</code>	
<code>(py-list (<Atom> ...))</code>	
<code>(py-tuple (<Atom> ...))</code>	
<code>(py-dict ((<Key> <Val>) ...))</code>	

2.6.3 Grounded Numeric and Relational Operators

Operator	Type
<code>+, -, *, /</code>	<code>(-> Number Number Number)</code>
<code><, <=, >, >=</code>	<code>(-> Number Number Bool)</code>
<code>==</code>	<code>(-> Atom Atom Bool)</code>

2.6.4 Console I/O

<code>(println! <Atom>)</code>	<code>(* prints to console *)</code>
<code>(trace! <Atom>)</code>	<code>(* debug output *)</code>

2.6.5 Assertions

<code>(assertEqual <Form> <Form>)</code>
<code>(assertEqualToResult <Form> (<Atom> ...))</code>

Error is a conventional symbol; assertions return () on success or an error atom on failure.

2.7 Advanced Features

2.7.1 Sealed Variables (Alpha-Conversion)

```
(sealed (<Variable> ...) <Atom>)
```

Renames all variables in the body except those listed, preventing accidental capture. Both the variable list and body are non-strict.

2.7.2 Syntactic Sugar (Optional)

These desugarings may be implemented in the parser:

```
(* Lambda arity sugar *)
(lambda2 $x $y body) => (lambda $x (lambda $y body))

(* List literals *)
(list a b c) => (Cons a (Cons b (Cons c Nil)))
```

2.7.3 Library definitions for if and quote (H-E MeTTa)

```
(: if      (-> Bool Atom Atom $t))
(= (if True  $then $else) $then)
(= (if False $then $else) $else)

(: quote (-> Atom Atom))
(= (quote $a) $a)
```

These are ordinary functions; their non-strict behavior stems from Atom-typed arguments.

2.8 Complete Grammar (EBNF)

```
Program      ::= { Toplevel } ;

Toplevel     ::= EvalItem | Form ;
EvalItem     ::= '!' Form ;

Form         ::= Atom | List ;

List         ::= '(' { Element } ')' ;
Element      ::= Atom | List ;

Atom         ::= Variable | Symbol | Grounded | Unit ;
Grounded     ::= Number | String | Bool ;
```



```

Variable      ::= '$' Symbol ;

Symbol        ::= SymStart { SymCont } ;
SymStart      ::= AsciiLetter | '_' | '+' | '-' | '*' | '/'
|
|           '<' | '>' | '=' | '%' | '!' | '?' | ':' |
|           '.' | '@' | '^' |
|           '~' | '|' | UnicodeLetter ;
SymCont       ::= SymStart | Digit ;

AsciiLetter   ::= 'A'..'Z' | 'a'..'z' ;
UnicodeLetter ::= <any Unicode code point whose
    General_Category?{Lu,Ll,Lt,Lm,Lo,Nl}
    and not in AsciiLetter> ;
Digit         ::= '0'..'9' ;

Whitespace    ::= U+0009 | U+000A | U+000D | U+0020 | <
    OtherUnicodeSpaces> ;
Comment       ::= ';' { AnyCharExceptLinebreak } ;
AnyCharExceptLinebreak ::= <any char except U+000A, U
    +000D> ;

String        ::= '"' { '\"' | '\n' | '\t' | '\\\\' |
    AnyExceptQuoteBackslash } '"' ;
AnyExceptQuoteBackslash ::= <any char except '"' or
    '\ '> ;

Number        ::= [ '+' | '-' ] Digit { Digit } [ '.'
    Digit { Digit } ] ;
Bool          ::= 'True' | 'False' ;
Unit          ::= '(' ')' ;

(* Patterns are syntactic; same terminals as Form but
    restricted *)
Pattern       ::= PatAtom | PatList ;
PatAtom       ::= Variable | Symbol | Number | String |
    Bool | Unit ;
PatList       ::= '(' { Pattern } ')' ;

(* Type expressions are ordinary Forms; accepted
    surface shapes *)
TypeExpr      ::= TypeAtom
| '(' '-'> TypeExpr { TypeExpr } ')'

```

```

      | '(' TypeExpr { TypeExpr } ')' ;
TypeAtom    ::= 'Type' | 'Atom' | 'Expression' | '
      Symbol'
      | 'Bool' | 'Number' | 'String' | 'Nat'
      | 'hyperon::space::DynSpace'
      | Symbol | Variable ;

(* Recognized heads (shapes) ? classification only *)
TypeDecl    ::= '(' ':' Pattern TypeExpr ')' ;
DefEq       ::= '(' '=' Pattern Form ')' ;
DefEqOneSide ::= '(' '=>' Pattern Form ')' ;
      optional extension

SymbolBind   ::= '(' 'bind!' Symbol Form ')' ;
Import       ::= '(' 'import!' Symbol Symbol ')' ;

(* Special forms with Atom (non-strict) positions
   preserved *)
LetForm      ::= '(' 'let' Pattern Form Atom ')' ;
LetStarForm  ::= '(' 'let*' '(' { '(' Pattern Form
      ')' } ')' Atom ')' ;
CaseForm     ::= '(' 'case' Form '(' { '(' Pattern
      Atom ')' } ')' ')' ;
MatchForm    ::= '(' 'match' Form Pattern Atom ')' ;
SuperposeForm ::= '(' 'superpose' '(' { Atom } ')' ')'
      ;
CollapseForm ::= '(' 'collapse' Atom ')' ;
SealedForm   ::= '(' 'sealed' '(' { Variable } ')'
      Atom ')' ;
PrognForm    ::= '(' 'progn' { Form } ')' ;

(* Interop and spaces (normative shapes) *)
NewSpace     ::= '(' 'new-space' ')' ;
AddAtom      ::= '(' 'add-atom' Form Atom ')' ;
RemoveAtom   ::= '(' 'remove-atom' Form Atom ')' ;
AddReduct    ::= '(' 'add-reduct' Form Form ')' ;
GetAtoms     ::= '(' 'get-atoms' Form Pattern ')'
      ;

PyAtom       ::= '(' 'py-atom' String [ TypeExpr ]
      ')' ;
PyDot        ::= '(' 'py-dot' Form Symbol ')' ;
Kwargs       ::= '(' 'kwargs' { '(' Symbol Form ')'
      } ')' ;

```

<pre> AssertEq ::= '(' 'assertEqual' Form Form ')' ; AssertEqRes ::= '(' 'assertEqualToResult' Form '(' { Atom } ')' ')' ; </pre>
--

2.9 Implementation Guidance

2.9.1 Parsing Pipeline

1. **Tokenization:** Convert character stream to tokens
2. **S-expression parsing:** Build tree structure
3. **Classification:** Identify designated forms and non-strict positions
4. **AST construction:** Tag non-strict positions for the transpiler

Scope note: classification may consult a fixed set of built-ins and library-declared shapes (e.g., functions whose types include `Atom`-typed parameters). In `H-E MeTTa`, `if` and `quote` are library functions; their laziness is derived from their types, not from parser-level special status.

2.9.2 Transpiler Requirements

The transpiler must:

- Preserve non-strict argument positions (marked as `Atom`)
- Expand tokens at parse time, not runtime
- Collect type declarations for optional checking
- Handle single-sided rules (or reject with diagnostic)

2.9.3 Why This Grammar Suffices for MeTTa-IL

- Simple S-expression base with recognized heads maps directly to IL nodes
- Non-strict positions are explicitly marked for correct compilation
- Token/space mechanics translate to IL declarations
- All extensions (sealed, single-sided rules, etc.) are expressible as ordinary lists with special interpretation

2.9.4 Minimal examples that parse under this grammar

```
; facts
(Parent Tom Bob)

; type and definition
(: inc (-> Number Number))
(= (inc $x) (+ $x 1))

; token and import
(bind! &kb (new-space))
(import! &self stdlib)

; query (non-strict output pattern)
! (match &self (Parent $x Bob) $x)

; control forms honoring non-strict positions
! (if (> 2 1) (quote OK) (quote FAIL))

; let/let*
! (let (age $a) (match &self (Age Alice $a) $a))

; nondeterminism
(= (coin) 0) (= (coin) 1)
! (collapse (coin)) ; materialize all results
into a tuple

; sealed (keep $x, freshen others)
! (sealed ($x) (Cons $x (Cons $y Nil)))

; grounded atoms: Python interop
(bind! &np (py-atom "numpy"))
! ((py-dot &np arange) (Kwargs (start 2) (stop 10) (
step 3)))

; spaces API
(add-atom &self (Male Tom))
! (get-atoms &self (Male $x))

; asserts and Error is just a symbol
! (assertEqual (+ 2 2) 4)
```

All of the above are legal by the grammar; behavior is determined by the standard library and the operational semantics in the IL/backend.

3 Compact Small-Step Operational Semantics for MeTTa

This section provides a complete small-step operational semantics for MeTTa aligned with the surface grammar. The semantics (i) matches practical usage including spaces, equality, nondeterminism, and control forms, and (ii) enables clean compilation to MeTTa-IL based on graph-structured λ -theories (GSLTs).

3.1 Overview and Approach

Style. We use a small-step SOS over a multi-register abstract machine from Meta-MeTTa, extended to cover all surface constructs needed for translation.

Evaluation model. Evaluation proceeds by query-driven rewriting via equalities. Nondeterminism is explicit. Grounded functions (primitives like numbers, strings, `+`, `and`, Python bridges) reduce by calling the host layer.

3.2 Abstract Syntax

We assume the surface grammar from Section 2. The runtime abstract syntax is:

Atoms and expressions.

$a ::= s \mid v \mid g \mid (a \ a \ \dots \ a)$
--

where s is a symbol, v a variable ($\$x$), g a grounded atom (numbers, strings, booleans, host objects), and $(a \ \dots)$ an application/expression.

Distinguished constructors (values).

- **Equalities:** $(= \ a \ a)$ and optionally $(=> \ a \ a)$ (directed, see §3.6.2)
- **Nondeterminism:** $(\text{superpose } (a \ \dots \ a))$, $(\text{collapse } a)$, (empty)
- **Sealing:** $(\text{sealed } (v_1 \ \dots \ v_k) \ a)$ for alpha-conversion on demand

Spaces and modules. A space atom Σ (e.g., `self`) denotes a MeTTa space; it is a grounded atom with the Space API.

Programs. A program is a multiset of atoms (facts and equalities) plus top-level requests `! \ a`. Type declarations $(: \ a \ T)$ are static and affect typing only (no reduction).

3.3 Machine State and Configuration

We adopt and extend the Meta-MeTTa four-register machine:

$$S = \langle I, K, W, O \mid \vartheta, \mathcal{M}, \nu \rangle$$

- I, K, W, O are multisets of terms (input, knowledge, workspace, output)
- ϑ is the *token map* mapping token refs (e.g., `np`) to atoms, mutated by `bind!` and `import!`
- $\mathcal{M}$ is the module store mapping module names to spaces or initial KBs
- ν is the freshness supply for generating unique variable names

A small step is $S \rightarrow S'$. The knowledge base K holds equalities (`= t u`) and facts (optionally tagged with spaces).

3.3.1 Freshness Service

The freshness supply ν provides globally unique variable names within a run.

Concurrency requirement. In concurrent executions, the freshness service must be *linearizable*: each allocation occurs at a single point in a total order consistent with real time. Implementations may use: (i) atomic counters, (ii) thread-safe PRNG/UUID sources, or (iii) serialized `sealed` steps. Programs must not rely on the spelling of fresh names, only their uniqueness.

3.3.2 Evaluation Conventions

Non-strict arguments. When a parameter has type `Atom`, the argument is *not reduced* before the function sees it. When the type is computational (e.g., `Number`, `Bool`), arguments are reduced first. This matches the behavior of `match`, `if`, `let`, `quote`, etc.

Insensitive predicate. For a term t and KB K , define:

$$\text{insensitive}(t, K) : \iff \forall (= t_i u_i) \in K. \text{unify}(t, t_i) \text{ fails}$$

For **Transform** with facts rather than equalities, use the same definition. The machine moves items between registers only when insensitive.

3.4 Core Reduction Rules

The core uses standard Meta-MeTTa rules with evaluation contexts $K[\cdot]$.

Query (apply equalities at the input).

$$\frac{\sigma_j = \text{unify}(t', t_j), \quad k = \{ (= t_1 u_1), \dots, (= t_n u_n) \} \uplus k', \quad \text{insensitive}(t', k')}{\langle \{ K[t'] \} \uplus I, k, W, O \rangle \rightarrow \langle I, k, \{ K[u_j \sigma_j] \mid j \in J \} \uplus W, O \rangle}$$

Chain (continue rewriting from workspace).

$$\frac{\sigma_j = \text{unify}(u, t_j), \quad k = \{ (= t_1 u_1), \dots, (= t_n u_n) \} \uplus k', \quad \text{insensitive}(u, k')}{\langle I, k, \{K[u]\} \uplus W, O \rangle \rightarrow \langle I, k, \{K[u_j \sigma_j] \mid j \in J\} \uplus W, O \rangle}$$

Transform (pattern-directed replacement).

$$\frac{\sigma_j = \text{unify}(t, t_j), \quad k = \{t_1, \dots, t_n\} \uplus k', \quad \text{insensitive}(t, k')}{\langle \{(\text{transform } t \ u)\} \uplus I, k, W, O \rangle \rightarrow \langle I, k, \{K_j[u \sigma_j] \mid j = 1..n\} \uplus W, O \rangle}$$

Space operations. We notate $a@ \Sigma$ as “atom a lives in space Σ ”.

$$\langle \{(\text{add-atom } \Sigma \ a)\} \uplus I, k, W, O \rangle \rightarrow \langle I, k \uplus \{a@ \Sigma\}, W, \{()\} \uplus O \rangle \quad (1)$$

$$\langle \{(\text{remove-atom } \Sigma \ a)\} \uplus I, \{a@ \Sigma\} \uplus k, W, O \rangle \rightarrow \langle I, k, W, \{()\} \uplus O \rangle \quad (2)$$

$$\langle \{(\text{remove-atom } \Sigma \ a)\} \uplus I, k, W, O \rangle \rightarrow \langle I, k, W, \{()\} \uplus O \rangle \quad (\text{absent OK}) \quad (3)$$

Get-atoms (space query). Let $\text{Facts}(\Sigma) = \{a \mid a@ \Sigma \in k\}$.

$$\frac{M = \{a\sigma \mid a \in \text{Facts}(\Sigma), \sigma = \text{unify}(a, p)\}}{\langle \{(\text{get-atoms } \Sigma \ p)\} \uplus I, k, W, O \rangle \rightarrow \langle I, k, W, \{(m_1 \dots m_m)\} \uplus O \rangle}$$

Output.

$$\langle I, k, \{u\} \uplus W, O \rangle \rightarrow \langle I, k, W, \{u\} \uplus O \rangle$$

3.4.1 Nondeterminism and Result Ordering

Multiple equalities may match, producing multiple results pushed to W then O .

Result ordering. Core semantics does not constrain the order of results from **superpose**, **collapse**, or **get-atoms**. Programs must not assume specific order.

Deterministic Profile (optional). Deployments requiring determinism may implement: (i) **get-atoms** orders matches lexicographically by printed form; (ii) **collapse** orders elements similarly. Engines advertising this profile must apply it uniformly.

3.5 Grounded Operations

Grounded operations reduce by host evaluation when arguments are values.

3.5.1 Basic Arithmetic and Logic

$$\langle \{ (+ \ n_1 \ n_2) \} \uplus I, k, W, O \rangle \rightarrow \langle I, k, W, \{ n_1 + n_2 \} \uplus O \rangle \quad (4)$$

$$\langle \{ (\text{and } b_1 \ b_2) \} \uplus I, k, W, O \rangle \rightarrow \langle I, k, W, \{ b_1 \wedge b_2 \} \uplus O \rangle \quad (5)$$

$$\langle \{ (== \ x \ y) \} \uplus I, k, W, O \rangle \rightarrow \langle I, k, W, \{ \text{True/False/BadType} \} \uplus O \rangle \quad (6)$$

3.5.2 Numeric and Relational Operators

The following are strict in all arguments:

Operator	Principal type
<code>+, -, *, /</code>	<code>(-> Number Number Number)</code>
<code><, <=, >, >=</code>	<code>(-> Number Number Bool)</code>
<code>==</code>	<code>(-> Atom Atom Bool)</code>

Invalid types yield `BadType` or typed `Error` (see §3.8.2). Engines MAY extend numeric towers but must document semantics.

3.5.3 Console I/O

```
(: println! (-> Atom Unit))
(: trace!   (-> Atom Unit))
```

Both are strict, perform console output (implementation-defined textualization), and return unit `()`. Formatting errors yield `(Error io-exn ...)`.

3.6 Surface Construct Semantics

Special forms are implemented via macro-expansion or dedicated rules.

3.6.1 Equality and Function Clauses

A clause `(= (f p1 ... pn) body)` contributes an equality to K . Evaluating `(f a1 ... an)` proceeds by Query/Chain via unification. This is the standard MeTTa “functions as equations” pattern.

3.6.2 Single-Sided Matching (Optional)

In addition to bidirectional `(= t u)`, we allow directed `(=> t u)`. Directed rules participate in Query/Chain but only left-to-right. Formally, maintain K_{eq} and $K_{\rightarrow}$; Query/Chain draw from $K_{\text{eq}} \cup K_{\rightarrow}$ using unification on the left side only.

3.6.3 Control Forms

match. Type: `(-> Space Atom Atom %Undefined%)`. Arguments 2 and 3 are non-strict.

$$\langle \{ (\text{match } \Sigma \ p_{in} \ p_{out}) \} \uplus I, k, W, O \rangle \rightarrow \langle I, k, W \uplus \{ p_{out} \sigma \mid \sigma \in \text{Solutions}(\Sigma \vdash p_{in}) \}, O \rangle$$

if. Only the condition is strict; branches are Atom-typed.

$$\langle \{(\text{if True } a \ b)\} \uplus I, k, W, O \rangle \rightarrow \langle \{a\} \uplus I, k, W, O \rangle \quad (7)$$

$$\langle \{(\text{if False } a \ b)\} \uplus I, k, W, O \rangle \rightarrow \langle \{b\} \uplus I, k, W, O \rangle \quad (8)$$

let and let*. $(\text{let } p \ e \ \text{body})$ evaluates e to v , unifies with pattern p , then evaluates $\text{body}\sigma$ on success:

$$\frac{e \Downarrow v \quad \sigma = \text{unify}(p, v)}{\langle \{(\text{let } p \ e \ \text{body})\} \uplus I, k, W, O \rangle \rightarrow \langle \{\text{body}\sigma\} \uplus I, k, W, O \rangle}$$

let* expands to nested **lets**.

case. $(\text{case } e \ ((p_1 \ r_1) \ \dots \ (p_n \ r_n)))$ evaluates e to v , finds first matching pattern p_i , returns $r_i\sigma$. Pattern **Empty** catches empty results.

Sequential composition: progn. $(\text{progn } e_1 \ \dots \ e_n)$ evaluates arguments left-to-right, returns last value. Strict in all arguments. Empty results or errors propagate.

$$(\text{progn } e) \rightarrow e \quad (\text{progn } e_1 \ e_2 \ \dots \ e_n) \rightarrow (\text{progn } e_2 \ \dots \ e_n) \text{ after } e_1 \Downarrow v$$

quote. $(\text{quote } a)$ creates a value preventing reduction of a until unwrapped.

3.6.4 Nondeterminism Operators

superpose.

$$\langle \{(\text{superpose } (a_1 \ \dots \ a_n))\} \uplus I, k, W, O \rangle \rightarrow \langle I, k, W, \{a_1, \dots, a_n\} \uplus O \rangle$$

collapse. Evaluates e and collects all results into a tuple:

$$\langle \{(\text{collapse } e)\} \uplus I, k, W, O \rangle \rightarrow \langle I, k, W, \{(r_1 \ \dots \ r_m)\} \uplus O \rangle$$

empty. Returns no result for branch pruning.

3.6.5 Expression Structure Operations

car-atom, **cdr-atom**, **cons-atom** operate on expressions at the meta level. Their **Expression**-typed arguments are non-strict unless explicitly bound via **let**.

3.6.6 Space API

self is the current program space. **(new-space)** creates a fresh space. **add-atom**, **remove-atom**, and **match** follow the core rules (§3.4). **(add-reduct Σ a)** adds the *reduced* form of a to Σ . **get-atoms** returns a tuple of matches.

3.6.7 Tokens and Modules

bind!

$$\frac{e \Downarrow v}{\langle \{(\text{bind! } \&t \ e)\} \uplus I, k, W, O \mid \vartheta, \mathcal{M}, \nu \rangle \rightarrow \langle I, k, W, \{()\} \uplus O \mid \vartheta[\&t := v], \mathcal{M}, \nu \rangle}$$

Future parsing replaces $\mathfrak{t}$ with v .

import!

$$\frac{\mathcal{M}(m) = K_m \quad \Sigma_m = \text{freshSpace}()}{\langle \{(\text{import! } \&t \ m)\} \uplus I, k, W, O \mid \vartheta, \mathcal{M}, \nu \rangle \rightarrow \langle I, k \uplus K_m @ \Sigma_m, W, \{()\} \uplus O \mid \vartheta[\&t := \Sigma_m], \mathcal{M}, \nu \rangle}$$

Loads module m into fresh space Σ_m bound to $\mathfrak{t}$.

Import cycles. Let $G = (\mathcal{M}, \mathcal{E})$ be the import graph. Programs are well-formed only if G is acyclic. Cycles reduce to:

$$(\text{import! } \&t \ m) \rightarrow (\text{Error import-cycle } m)$$

Optional: Engines may support cycles by allocating spaces for SCCs first.

3.6.8 Sealed (Alpha-Conversion)

(sealed (v₁ ... v_k) t) renames all variables in t except $\{v_1, \dots, v_k\}$ to fresh names:

$$\langle \{(\text{sealed } (v_1 \dots v_k) \ t)\} \uplus I, k, W, O \mid \vartheta, \mathcal{M}, \nu \rangle \rightarrow \langle \{\alpha_\rho(t)\} \uplus I, k, W, O \mid \vartheta, \mathcal{M}, \nu' \rangle$$

where ρ maps unrestricted variables to fresh names from ν .

3.7 Python Interoperability

Three grounded primitives with host evaluation abstracted by `pyEval` and `pyAttr`.

py-atom. Returns a grounded object with optional type:

$$\langle \{(\text{py-atom } s \ [T])\} \uplus I, k, W, O \rangle \rightarrow \langle I, k, W, \{\text{pyEval}(s, T)\} \uplus O \rangle$$

py-dot. Attribute access:

$$\langle \{(\text{py-dot } g \ a)\} \uplus I, k, W, O \rangle \rightarrow \langle I, k, W, \{\text{pyAttr}(g, a)\} \uplus O \rangle$$

Grounded call with keywords.

$$\langle \{(g \ \vec{v} \ \text{Kwargs } \text{Kw})\} \uplus I, k, W, O \rangle \rightarrow \langle I, k, W, \{\text{pyEval}(g, \vec{v}, \text{Kw})\} \uplus O \rangle$$

3.7.1 Interop Lifecycle

Object lifetime. Grounded objects must be reclaimable when unreferenced (via GC/RC/finalizers). Cross-runtime cycles need weak references or explicit release.

Host exceptions. Host boundary exceptions reduce to error atoms:

$$\rightarrow (\text{Error py-exn } \langle \text{op} \rangle \langle \text{message} \rangle [(\text{host-type})])$$

Type mapping. Map host scalars: Python `bool`→`Bool`, `int/float`→`Number`, `str`→`String`. Containers via `py-list/py-tuple/py-dict` or wrapped consistently.

3.8 Error Handling

3.8.1 Assertions

`Error` is a conventional symbol; no runtime exceptions exist.

assertEqual. Let $\llbracket e \rrbracket$ be the multiset of results of e :

$$\langle \{(\text{assertEqual } e_1 \ e_2)\} \uplus I, k, W, O \rangle \rightarrow \begin{cases} \langle I, k, W, \{()\} \uplus O \rangle & \text{if } \llbracket e_1 \rrbracket \equiv \llbracket e_2 \rrbracket \\ \langle I, k, W, \{(\text{Error } \text{assertEqual } e_1 \ e_2)\} \uplus O \rangle & \text{otherwise} \end{cases}$$

assertEqualToResult. Compares $\llbracket e \rrbracket$ to a literal tuple $(r_1 \ \dots \ r_m)$.

3.8.2 Error Model and Propagation

Errors are first-class atoms: $(\text{Error tag } a_1 \ \dots \ a_k)$.

Propagation. Strict arguments with errors fail fast:

$$f(\dots, (\text{Error } \dots), \dots) \rightarrow (\text{Error propagate } f \ \dots)$$

Non-strict arguments carry errors unevaluated until forced.

Matching. Errors are matchable data. Libraries can define `is-error` and `raise` helpers.

Lazy evaluation. Forcing an error propagates it:

$$(\text{force } (\text{Error } \dots)) \rightarrow (\text{Error propagate force } \dots)$$

3.9 Process Interpretation (Concurrency)

Spaces can be viewed as channels for process-algebraic reading:

- Facts $a@Σ$ act as persistent inputs
- `add-atom` installs facts
- `remove-atom` withdraws guards
- `match` enumerates synchronizations

This justifies compilation to RSpace or similar concurrent backends.

3.10 Static Typing (Optional)

Type declarations $(\vdash a : T)$ populate environment Δ but don't reduce. Dynamic type checks in grounded calls yield `BadType` or `Error` atoms on mismatch.

3.11 Transpilation to MeTTa-IL

The semantic rules map directly to IL constructs:

Sorts. T (terms), R (rewrites), Space, Ground.

Term constructors. Application, Equality, Quote, Superpose, Collapse, Empty, Space ops, Sealed, Python interop.

Rewrite edges. Query, Chain, Transform, Add, Remove, Get, Output, grounders, Seal.

Strictness preservation. Non-strict positions for `if`, `match`, `let`, `case`, `quote`, `superpose`, `collapse`, `sealed`.

Environment. Tokens and imports become IL declarations updating token tables and spaces.

Nondeterminism. Multiple edges for `superpose`; frontier collection for `collapse`.

3.12 Worked micro-examples (sanity checks)

Function by equality.

```
(: inc (-> Number Number))  
(= (inc $x) (+ $x 1))  
! (inc 2)
```

`(inc 2)` rewrites to `(+ 2 1)` then to 3.

Pattern match over a space.

```
(Female Pam)
(Parent Tom Bob)
! (match &self (Parent $x Bob) $x)
```

match queries self, emits each x .

superpose / collapse dual.

```
(= (color) green) (= (color) red) (= (color) yellow)
! (collapse (color)) ; => (green red yellow) as one
  result
```

Sealed to avoid capture.

```
! (sealed ($x) (Cons $x (Cons $y Nil))) ; freshens
  $y, keeps $x
```

Python interop with kwargs.

```
(bind! &np (py-atom "numpy"))
! ((py-dot &np arange) (Kwargs (start 2) (stop 10) (
  step 3)))
```

remove-atom absent is OK.

```
! (remove-atom &self (Foo Bar)) ; always yields ()
```

3.13 Design Decisions and Non-Assumptions

- **No fixed λ -calculus:** MeTTa uses equations and unification. Lambda-like sugars are grounded closures.
- **Backend-agnostic concurrency:** Space rules compile to various concurrent backends.
- **Evaluation order:** Not customizable here (see Section 5 for that extension).

3.14 Summary for Transpiler Implementers

Classify each surface construct as:

- **Kernel:** Equality rewriting, match, space ops, nondeterminism, quote, sealed, interop $\rightarrow$ IL rewrites
- **Macro:** let, let*, case, if $\rightarrow$ preserve Atom discipline
- **Environment:** Tokens/modules $\rightarrow$ IL declarations and space constants

Preserve nondeterminism as multiple edges, implement **collapse** as tuple collection, follow Meta-MeTTa bisimulation for correctness.

4 Alternative Presentation: MeTTa Operational Semantics via the MeTTa Calculus

The prior section suffices as a formal presentation of MeTTa semantics, however, it can in some ways be an unwieldy way of looking at things. This section recasts the small-step register-machine semantics given above as reduction in the *MeTTa Calculus* [3], which is in some ways a simpler and more elegant presentation. Where the machine speaks in terms of four registers (I, K, W, O) and rules (Query, Chain, Transform, Add/Remove/Get, Output), the calculus speaks in terms of processes, names (spaces), and a single synchronization rule (COMM) acting on tagged atoms. The two views are equivalent up to structural congruence and permutation of independent steps; this section gives a concrete translation and correspondence.

4.1 MeTTa Calculus fragments used here

We assume the following standard fragments. Processes $P, Q ::= 0 \mid G \mid \text{for}(t \leftrightarrow x)P \mid x?P \mid !P \mid P \mid Q \mid \text{new } x \text{ in } \{P\}$, names $x, y ::= @P$, and atoms/terms $t, u ::= \text{atom} \mid ([t])$. Reduction is generated by structural congruence, parallel, and the single-step synchronization (COMM):

$$\frac{\sigma = \text{unify}(t, u)}{\text{for}(t \leftrightarrow x)P \mid \text{for}(u \leftrightarrow x)Q \longrightarrow P\sigma^\bullet \mid Q\sigma^\bullet} \quad (\text{COMM})$$

where $\sigma^\bullet$ replaces variable $\rightarrow$ process bindings with variable $\rightarrow$ name bindings. Intuitively, $\text{for}(t \leftrightarrow x)P$ offers a guarded continuation at “space/tag” x that fires when some u unifies with t .

4.2 Distinguished spaces and an “emit” macro

We use four distinguished names to mirror the machine registers: I, K, W, O . We write

$$\text{emit}(X, a) \triangleq \text{for}(a \leftrightarrow X) 0$$

to “tag” an atom a into the space X .³ Uninterpreted ground effects (I/O, host calls) are treated as ground processes G .

4.3 Compilation of program items to processes

Let $\llbracket \cdot \rrbracket$ produce a calculus process.

Facts and equalities in K . A fact $a@ \Sigma$ is compiled to a (by default, persistent) offer at Σ :

$$\llbracket a@ \Sigma \rrbracket = ! \text{emit}(\Sigma, a).$$

³As in the calculus, “spaces = tags”: membership is represented by the presence of guarded offers at the space/tag, not a container structure.

A bidirectional equality $(= t u)$ yields two rewrite offers anchored at a distinguished rule space **Eq**, each re-emitting its right-hand side to the workspace:

$$\llbracket (= t u) \rrbracket = ! (\text{for}(t \leftrightarrow \text{Eq}) \text{emit}(\text{W}, u)) \mid ! (\text{for}(u \leftrightarrow \text{Eq}) \text{emit}(\text{W}, t)).$$

Directed rules $(=> t u)$ are compiled with only the first clause.

Top-level requests A request injects e into the input and posts a driver that asks for its value:

$$\llbracket ! e \rrbracket = \text{emit}(\text{I}, e) \mid \text{for}(e \leftrightarrow \text{I}) \text{for}(e \leftrightarrow \text{Eval}) 0,$$

where **Eval** is the evaluation space (see **Query/Chain** below). Other top-level space operations are compiled as below.

4.4 Reading the machine rules through **Comm**

Query/Chain. The machine steps *Query* and *Chain* (Sec. 3.4) rewrite a redex against equalities in K , pushing results to W . In the calculus, we put redexes in **Eval** and rules in **Eq**:

$$\underbrace{\text{for}(t' \leftrightarrow \text{Eval}) 0}_{\text{the redex}} \mid \underbrace{\text{for}(t \leftrightarrow \text{Eq}) \text{emit}(\text{W}, u)}_{\text{the rule}} \implies \text{emit}(\text{W}, u\sigma), \quad \sigma = \text{unify}(t', t).$$

Thus a **COMM** step *is* **Query/Chain**; repeated use yields the usual rewrite closure into W .

Transform. Machine *Transform* matches a pattern t against facts in a space Σ and emits $u\sigma$. Compile facts as $! \text{emit}(\Sigma, a)$ and the transform request as $\text{for}(t \leftrightarrow \Sigma) \text{emit}(\text{W}, u)$. A single **COMM** produces $\text{emit}(\text{W}, u\sigma)$.

Output and “insensitive”. The machine moves an item from W to O when it is *insensitive* to K (no left-hand side unifies). In the calculus, this is realized by a policy: drivers only re-emit to **O** after no further **Eq**-synchronization applies. Formally this is a fairness/strategy condition; observationally, both models output the same normal forms up to permutation.

4.5 Special forms and library heads

All surface heads are realized as ordinary rewrite programs over **Eq**. In particular (non-strict positions preserved by using atoms as data):

$$\begin{aligned} &(\text{if True } a \ b) \Rightarrow a, & (\text{if False } a \ b) \Rightarrow b, \\ &(\text{let } p \ e \ a) \Rightarrow a\sigma & \text{when } e \Rightarrow v, \ \sigma = \text{unify}(p, v), \\ &(\text{case } e \ ((p_i \ r_i) \dots)) \Rightarrow r_j\sigma & \text{for the first matching } p_j, \\ &(\text{match } \Sigma \ p \ r) \Rightarrow r\sigma & \text{via } \text{for}(p \leftrightarrow \Sigma) \text{emit}(\text{W}, r). \end{aligned}$$

Each clause is installed as $\text{for}(_ \leftrightarrow \text{Eq})$ -offers that re-emit to W , exactly as above.

Nondeterminism. `superpose( $a_1, \dots, a_n$ )` compiles to parallel emissions `emit( $W, a_1$ ) | \dots | emit( $W, a_n$ )`. `collapse  $e$` uses fork-join: spawn the branches of e and collect their results into a tuple using a fresh collector name c and a join-pattern that waits for the end-of-stream marker (omitted here for brevity); the result is emitted to W . Either way, COMM explores the same nondeterministic frontier as the machine.

Expression-structure ops. `car-atom`, `cdr-atom`, `cons-atom` are ordinary grounders; they appear as ground processes G in the calculus and simply emit their result.

4.6 Spaces, modules, and tokens

Space API. `new-space` is `new  $\Sigma$  in  $\{0\}$` and returns Σ . `add-atom  $\Sigma a$` installs (by policy) a persistent offer `! emit( $\Sigma, a$ )`. `remove-atom  $\Sigma a$` withdraws one such guard; operationally this is a COMM with a non-replicated retractor (or, in a capability discipline, consumes one token authorizing the guard), returning unit. `get-atoms  $\Sigma p$` is just `for( $p \leftrightarrow \Sigma$ )`-driven emission.

bind!/import!. `bind!  $\&t e$` evaluates e to a name/value and thereafter uses that name as the token for a space or grounded object; in the calculus this is a local definition of a name (possibly via `new`). `import!` installs the imported module's facts/rules as offers at a fresh space bound to $\&t$.

4.7 Sealed (alpha-conversion)

`sealed( $(v_1 \dots v_k) t$ )` freshens all variables in t except those listed. In the calculus this is realized by generating fresh names via `new` and systematically renaming inside the quoted body before emission. Freshness linearizability follows from name generation discipline; this mirrors the machine's freshness service.

4.8 Grounded numerics, interop, and I/O

Numeric/relational heads are deterministic grounders G that, when their arguments are values, emit a ground value. Python interop heads (`py-atom`, `py-dot`, `keyword-call`) are likewise grounders bridging to the host; exceptions/BadType become ordinary `Error` atoms that flow like data.

4.9 Errors and assertions

`Error` is an atom; strict positions propagate it eagerly (a rewrite to `Error`), non-strict positions pass it as data until forced. Assertions are rewrites that compare (multi)sets of results and emit unit on success or an `Error` atom otherwise.

4.10 Concurrency and transactional reading

Placing facts/rules as space-tagged guards and using COMM gives a process interpretation of the machine's concurrent reading of spaces: facts act as persistent inputs; adds install guards; removes withdraw them. Each COMM is a tiny transaction in which both sides read from their peer and write their continuations, exactly matching the operational intent.

4.11 Correspondence (bisimulation up to scheduling)

Write *Conf* for machine configurations and *Proc* for calculus processes. Define $Tr : Conf \rightarrow Proc$ by compiling I, K, W, O into offers at I, K, W, O and installing the driver policies described above. Then:

Theorem 1 (Step correspondence). *For every machine step $S \rightarrow S'$ there is a finite calculus reduction $Tr(S) \Longrightarrow^* Tr(S')$. Conversely, every calculus COMM step arising from an Eq or space synchronization corresponds to a machine Query/Chain/Transform step on the same redex/rule/fact.*

Sketch. Query/Chain/Transform are single COMM events between Eval/Eq or Σ -tagged offers; their continuations re-emit into W, matching the machine's target register. Grounders/I/O are local steps in both models. Nondeterminism is realized by parallel composition of independent COMMS, whose result ordering is unspecified in both models. Output/insensitive coincides under the normal-form emission policy. $\square$

Takeaway. The calculus refactors the machine rules down to one symmetric interaction law. Equalities, space queries, nondeterminism, sealing, interop and errors are all expressed as ordinary guarded offers on names, with the same observable results as the register-machine semantics.

5 MeTTa-IL: A Concrete Intermediate Language Schema

This section provides a concrete MeTTa-IL (IL) schema suitable for transpilers and execution backends. It defines the textual program format used by the BNFC-based reference parser, the term constructors with types and strictness, and the edge (rewrite) families implementing the small-step semantics from Section 3.

5.1 Motivation and Scope

The MeTTaCycle architecture document describes IL from a systems perspective, showing its place in runtime modules and the broader AGI stack. This section serves a complementary language role:

Machine rule / op	MeTTa Calculus (COMM instance)	Effect / tar- get
<i>Query</i> ($I \rightarrow W$ via K)	$\text{for}(t' \leftrightarrow \text{Eval}) 0 \mid \text{for}(t \leftrightarrow \text{Eq}) \text{emit}(W, u) \rightarrow \text{emit}(W, u\sigma)$, where $\sigma = \text{unify}(t', t)$	push to W
<i>Chain</i> (W closure via K)	$\text{for}(u \leftrightarrow \text{Eval}) 0 \mid \text{for}(t \leftrightarrow \text{Eq}) \text{emit}(W, v) \rightarrow \text{emit}(W, v\sigma)$	keep in W
<i>Transform</i>	$\text{for}(t \leftrightarrow \Sigma) \text{emit}(W, u) \mid \text{for}(a \leftrightarrow \Sigma) 0 \rightarrow \text{emit}(W, u\sigma)$, $\sigma = \text{unify}(t, a)$	pattern on space Σ
<i>add-atom</i>	install a persistent offer: $! \text{emit}(\Sigma, a)$	fact in Σ
<i>remove-atom</i>	consume a non-replicated retractor against $\text{emit}(\Sigma, a)$	withdraw 1 fact
<i>get-atoms</i>	enumeration via $\text{for}(p \leftrightarrow \Sigma) \text{emit}(W, p)$ firing against facts in Σ	tuple in O
<i>Output</i> (<i>insensitive</i>)	policy: re-emit $w \in W$ to O once no Eq -sync applies	move $W \rightarrow O$
<i>Nondet. superpose</i>	parallel emissions: $\text{emit}(W, a_1) \mid \dots \mid \text{emit}(W, a_n)$	many results
<i>collapse</i>	fork?join sugar: collect branch results to one tuple, then $\text{emit}(O, (r_1 \dots r_m))$	1 tuple
<i>sealed / freshness</i>	new x in $\{P\}$ for fresh names; rename inside guarded body before emit	hygienic re-name
<i>bind!/import!</i>	bind fresh name to space/object; imported rules become offers at a bound space	env update

Figure 1: Register-machine rules (Sec. 3) as single or small families of MeTTa-Calculus COMM steps acting on tags-as-spaces. Equalities live at **Eq**; redexes at **Eval**; facts/rules at named spaces Σ ; results flow to W then O .

- Surface-to-IL bridge: shows exactly how constructs like **let**, **case**, **match**, **sealed**, and **progn** lower to IL forms.
- Formal semantics: defines unification success and failure, strictness rules, memoization scope for by-needed arguments, and error propagation.
- Conformance criteria: provides a normative reference for engine implementers to ensure alignment.
- Tool support: gives precise, language-level descriptions for programmers and tool builders.

5.2 BNFC Textual IL Conventions

The reference implementation parses IL with a BNFC-based textual grammar. We adopt the following conventions so examples here are runnable:

- A program file is a sequence of items (declarations, rules, commands), followed by a single period . that terminates the file.

- Top-level commands use leading directives: `! e` to evaluate an expression `e`, and `? e` to run a query expression `e`.
- S-expressions have the generic form `(head arg1 ... argn)`. The first element is the head symbol by convention.
- The head for removing an atom is `rem-atom` (not `remove-atom`).
- Boolean literals are `true` and `false`. Type names use `Bool`, `Long`, `String`, etc.

These textual conventions are presentation-level; the operational rules below are independent of any specific parser.

5.3 Design Goals

- Executable: every IL program has precise small-step semantics mirroring the register machine.
- Transpilable: 1:1 lowering from surface MeTTa with no ambiguity about strictness or contexts.
- Host-agnostic: grounded calls (arithmetic, Python interop) are abstracted; engines provide host implementations.
- Modular: spaces and tokens are explicit at the IL level via ordinary terms and top-level commands.

5.4 Venus front-end pipeline (Modules to Graph IR)

To give the reader some understanding of the software context in which Metta-IL is operating, we summarize here the end-to-end pipeline for the *Venus* module language (the non S-expression DSL used in files with extension `.module`), and clarifies how it feeds the same Abstract IL / GSLT graph used by backends. This information regarding the code in the F1R3FLY GitHub repo is not strictly needed for understanding the MeTTa language, but is interesting for those who want to understand the software practicalities of the compilation of MeTTa into Metta-IL and then into operations on MORR and rholang and so forth.

Input. A Venus module is a BNFC-defined text format with top-level `Module` and nested `Theory` declarations. The language provides blocks for `Exports`, `Terms`, `Equations`, `Replacements`, and `Rewrites`, plus theory instantiation and composition forms. This concrete syntax is distinct from the S-expression IL, but both target the same graph IR.

Pass structure. The front-end executes a small sequence of named passes over a shared context. Conceptually:

```
Pass 1: LoadModules
- Parse the entry module (.module) and all imports
- Build a module-level index of theory declarations/
  instantiations

Pass 2: FindFinalInst
- Select the final TheoryInst in the entry module as
  the pipeline root

Pass 3: CheckInterpret
- Run well-formedness checks on the selected
  TheoryInst:
* label/category lookup
* variable-category consistency
* free-variable hygiene

Pass 4: Interpret
- Evaluate the TheoryInst into a BasePres (base
  presentation):
* resolve imports, replacements, exports
* accumulate terms, equations, rewrites into a
  typed presentation

Pass 5: DesugarBinders
- Lower binder sugar to explicit arrow categories
- Add desugared rules with mangled labels for
  canonical processing

Pass 6 (optional): Hypercube
- Type-lift arrow/product categories into a first-
  order shape
- Extend rules with parameters for repeated LHS
  variables

Pass 7: GenerateBNFC (presentation rendering)
- Monomorphize arrow/product categories into
  concrete labels
- Emit a normalized presentation where higher-order
  cats are flattened
```

Up to and including **BasePres**, we are still in the theory/presentation layer. The next step constructs the graph-structured IL (GSLT) used by backends.

BasePres $\rightarrow$ **Graph IR (GSLT)**. After **Interpret** (and optional **Hypercube**), the front-end possesses a typed, desugared **BasePres** containing:

- a finite signature of labels and their categories,
- a set of equations and directed rewrites with consistent sorts,
- replacements/exports resolved, and binders made explicit.

The graph builder then produces the GSLT graph:

- nodes for term constructors, categories, and space parameters,
- edges for rewrite families (Query/Chain/Transform, If/Let/Case, Match/Add/Rem, Superpose/Collapse, etc.),
- per-head strictness and non-strict contracts as edge context rules,
- optional policy metadata (evaluation policy, sealed freshness).

This *Graph IR* is the input to backends such as the Rholang emitter. In Section 5.8 we give the small-step rules on the graph-level IL; both textual front-ends compile to this same form.

Why two textual front-ends? The toolchain supports:

1. the **Venus module language** (this subsection), oriented to theory construction and algebraic presentations; and
2. the **BNFC S-expression IL** (Section 5.2), oriented to a Lisp-like concrete IL with top-level `!/?`.

They are alternative concrete syntaxes that compile to the same Abstract IL / GSLT graph. The operational semantics and conformance requirements apply at the graph level.

Takeaway for implementers. Engines and backends need only implement the graph IR. Front-end authors are free to accept either Venus modules or S-expression IL (or both), so long as they lower to the graph with the strictness and rewrite families specified in Sections 5.8 and 5.9.

Takeaway for this paper. For the purposes of this paper, as noted above, it seems OK to present MeTTa-IL using the BNFC S-expression syntax (Section 5.2) as we do in the worked examples in Section 5.13. The reference toolchain does include a non S-expression front-end (the Venus module language), but both front-ends compile to the same Abstract IL / GSLT graph that our formal semantics targets. In other words:

- **S-expression IL** is an accepted, runnable concrete format for IL programs (with top-level `!/?` items and a final `.`); it is suitable for examples and interchange.

- **Venus modules** provide a more explicitly-typed, theory-oriented concrete format; they are also compiled to the same graph IR before execution or code generation.

Readers should distinguish the *textual* IL (S-expr or Venus) from the *Abstract IL / Graph IR*, which is the common, typed execution model used by all backends. The small-step rules in this paper are defined for the graph-level IL, independent of whichever textual front-end a tool or programmer prefers.

5.5 Textual Program Format (BNFC-compatible)

A MeTTa-IL program file is a list of items followed by a period:

```

Program ::= { Item } "."
Item    ::= KItem | TopCmd

TopCmd  ::= "!" Expr          ; evaluate expression
          | "?" Expr          ; query expression

KItem   ::= TypeDecl          ; (: f T)
          | Eq                ; (= lhs rhs)
          | EqDir              ; (=> lhs rhs)
          | BindSym            := "(bind! " Sym " " Expr ")"
          | ImportSym          := "(import! " Sym " " Sym ")"
          | SpaceOp            ; (add-atom S a), (rem-atom S a), (match S p r)
          | FactAt              ; (fact@ S a) ;
                                optional, if supported

TypeDecl ::= "(: " Sym Type ")"
Eq        ::= "(= " Term " " Term ")"
EqDir     ::= "(=> " Term " " Term ")"

SpaceOp   ::= "(add-atom " Space " " Term ")"
          | "(rem-atom " Space " " Term ")"
          | "(match " Space " " Atom " " Atom ")"

BindTok   ::= "(bind! " Tok " " Expr ")"
ImportTok ::= "(import! " Tok " " Sym ")"

```

Notes: (i) **Space** is typically introduced by binding a symbol to (**new-space**) via (**bind! S (new-space)**). (ii) The grammar admits generic S-expressions (**head ...**); the semantics below assigns meaning to the designated heads.

5.6 Core Structure (Abstract)

5.6.1 Sorts and Base Types

We use the following abstract sorts in the semantics:

Sym, Var, Space, Ty, Ground, Term, Tuple, Bool, Num, Str, Unit

plus a meta-level multiset type for machine registers.

5.7 Term Constructors and Strictness

The following constructors are referenced by the semantics. "Strict" means the argument is evaluated before the rule fires. "Atom" means the argument is passed unreduced.

5.7.1 Core Data and Application

(sym : String -> Sym)	; symbols
(var : String -> Var)	; variables (
patterns only)	
(num : Int64 -> Num)	; numbers
(str : String -> Str)	; strings
(bool : Bool -> Bool)	; booleans (true/
false literals)	
(unit : -> Unit)	; unit value
(tuple : [Term] -> Tuple)	; tuples
(app : Term {Term} -> Term)	; application (
strict)	
(quote : Term -> Term)	; quotation (Atom
)	

5.7.2 Equations and Rules

(= : Term Term -> Term)	; bidirectional
equality	
(=> : Term Term -> Term)	; directed (
forward-only) rule	

5.7.3 Control Forms

(if : Bool Atom Atom -> Term)	; strict: 1, non-
strict: 2,3	

```

(let      : Pat  Term Atom -> Term)      ; strict: 2, non-
      strict: 1,3
(case    : Term [(Pat Atom)] -> Term)    ; strict:
      scrutinee
(transform : Term Term -> Term)          ; both strict
(progn   : {Term} -> Term)                ; strict left-to-
      right, returns last

```

5.7.4 Nondeterminism

```

(superpose : Tuple -> Term)              ; Atom: elements
      unreduced
(collapse  : Term -> Term)                ; Atom: argument
      unreduced
(empty     :          -> Term)            ; yields no
      result

```

5.7.5 Spaces and Queries

```

(match      : Space Atom  Atom -> Term) ; non-strict:
      2,3
(add-atom   : Space Atom  -> Term)       ; Atom: added
      as-is, returns unit
(rem-atom   : Space Atom  -> Term)       ; returns unit
      (even if absent)
(get-atoms  : Space Pat   -> Term)       ; returns tuple
(add-reduct: Space Term   -> Term)       ; strict: 2 (
      adds reduced form)

```

5.7.6 Sealing and Interop

```

(sealed : [Var] Atom -> Term)            ; Atom: body;
      freshens vars not listed

(py-atom : Str [Ty] -> Ground)           ; import a Python
      object
(py-dot   : Ground Sym -> Ground)        ; attribute
      access
(kwargs   : [(Sym Term)] -> Term)        ; keyword
      arguments bundle

```


5.7.7 Assertions and Diagnostics

```
(assertEq  : Term Term  -> Term)      ; compare result
  sets
(assertEqR : Term Tuple -> Term)      ; compare to
  literal tuple
(println!  : Atom  -> Unit)           ; console print (
  strict)
(trace!    : Atom  -> Unit)           ; console trace (
  strict)
```

5.7.8 Patterns

Patterns appear in `=`, `=>`, `let`, `case`, `match`, and `get-atoms`:

```
Pat ::= (p-var <string>) | (p-sym <string>) | (p-num <
  int>)
      | (p-str <string>) | (p-bool <bool>) | (p-unit)
      | (p-space <sym>)  | (p-app Pat {Pat})
```

5.8 Operational Semantics (Edge Families)

We use the Meta-MeTTa style four-register machine $\langle I, K, W, O \rangle$ and small-step rules. The predicate $\text{insensitive}(t, K)$ is defined as:

$$\text{insensitive}(t, K) : \Longleftrightarrow \forall (= t_i u_i) \in K. \text{unify}(t, t_i) \text{ fails.}$$

5.8.1 Core Rewriting

Query and Chain. `E.Query` applies equalities from input: unify redex t' with each left-hand side and produce $u_i\sigma$ for all unifiers σ , provided $\text{insensitive}(t', K)$. `E.Chain` is identical on workspace items.

Transform. `E.Transform`: $(\text{transform } t \ u)$ matches t against facts and emits $u\sigma$ for each match.

5.8.2 Spaces

- `E.Add`: $(\text{add-atom } \Sigma \ a) \rightarrow ()$, adds $a@ \Sigma$ unreduced.
- `E.AddReduct`: $(\text{add-reduct } \Sigma \ e) \rightarrow ()$, adds the reduced value of e .
- `E.Remove`: $(\text{rem-atom } \Sigma \ a) \rightarrow ()$, removes one $a@ \Sigma$ if present.
- `E.Get`: $(\text{get-atoms } \Sigma \ p) \rightarrow (m_1 \dots m_k)$ with $m_i = a\sigma$.
- `E.Match`: $(\text{match } \Sigma \ p_{in} \ p_{out}) \rightarrow p_{out}\sigma$ for each unifier σ .

5.8.3 Control Flow

- E.IfTrue / E.IfFalse: branch on boolean condition.
- E.Let: evaluate e , unify with p , reduce to $a\sigma$ on success; otherwise empty.
- E.Case: sequential first-match semantics.
- E.Progn: strict left-to-right evaluation; result is the last value.
- E.Quote: value constructor that prevents inner evaluation.

5.8.4 Nondeterminism

- E.Superpose: (*superpose* $(a_1 \dots a_n)$) yields n successors.
- E.Collapse: collects all results of e into a single tuple.
- E.Empty: produces no result.

5.8.5 Sealed, Output

- E.Seal: alpha-conversion using a linearizable freshness service ν .
- E.Output: move an insensitive item from W to O .

5.8.6 Python Interop

- E.PyAtom: (*py-atom* $s [T]$) $\rightarrow$ grounded object.
- E.PyDot: (*py-dot* $g a$) $\rightarrow$ attribute value.
- E.GCall: calling a grounded value with positional and keyword args.
- Host exceptions map to (*Error py - exn ...*) and do not mutate K or tokens.

5.8.7 Assertions and Errors

- E.AssertEq: compare multisets of results; return unit or (*Error...*).
- E.AssertEqr: compare to a literal tuple (treated as a value).

5.9 Lowering from Surface to IL

Let $\mathcal{L}[\cdot]$ translate surface terms and $\mathcal{P}[\cdot]$ translate patterns. Lowering preserves non-strict positions.

5.9.1 Pattern Translation

$$\begin{aligned}\mathcal{P}[\$x] &= (p\text{-}var \text{"x"}) \\ \mathcal{P}[s] &= (p\text{-}sym \text{"s"}) \\ \mathcal{P}[n] &= (p\text{-}num \ n) \\ \mathcal{P}[(t_0 \ \dots \ t_k)] &= (p\text{-}app \ \mathcal{P}[t_0] \ \dots \ \mathcal{P}[t_k])\end{aligned}$$

5.9.2 Control Forms

Let.

$$\mathcal{L}[(let \ p \ e \ a)] = (let \ \mathcal{P}[p] \ \mathcal{L}[e] \ \mathcal{L}[a])$$

IL step: evaluate e , unify with p ; on success reduce to $a\sigma$; on failure produce no result.

Case.

$$\mathcal{L}[(case \ e \ ((p_1 r_1) \dots))] = (case \ \mathcal{L}[e] \ ((\mathcal{P}[p_1] \ \mathcal{L}[r_1]) \dots))$$

Progn (sequential composition).

$$\mathcal{L}[(progn \ e_1 \dots e_n)] = \begin{cases} \mathcal{L}[e_1] & n = 1 \\ (let \ (p\text{-}var \text{"_tmp"}) \ \mathcal{L}[e_1] \ \mathcal{L}[(progn \ e_2 \dots e_n)]) & n > 1 \end{cases}$$

5.9.3 Other Forms

$$\begin{aligned}\mathcal{L}[(match \ \Sigma \ p \ r)] &= (match \ \mathcal{L}[\Sigma] \ \mathcal{P}[p] \ \mathcal{L}[r]) \\ \mathcal{L}[(quote \ a)] &= (quote \ \mathcal{L}[a]) \\ \mathcal{L}[(sealed \ (v_1 \dots) \ t)] &= (sealed \ ((var \ \text{"v1"}) \dots) \ \mathcal{L}[t])\end{aligned}$$

5.9.4 Grounded Operators

Grounded calls lower to ordinary applications:

$$\mathcal{L}[(+ \ e_1 \ e_2)] = (app \ (sym \ \text{"+"}) \ \mathcal{L}[e_1] \ \mathcal{L}[e_2])$$

Reduction is handled by grounder edge families (numeric, relational, I/O).

5.10 Strictness Table

Non-strict positions for special forms:

Constructor	Non-strict positions
<code>if</code>	2, 3 (branches)
<code>let</code>	1, 3 (pattern, body)
<code>case</code>	branch results
<code>match</code>	2, 3 (pattern and result)
<code>quote</code>	1 (body)
<code>superpose</code>	tuple elements
<code>collapse</code>	1 (expression)
<code>sealed</code>	1 (var list), 2 (body)
<code>assertEqr</code>	2 (literal tuple)
<code>progn</code>	none (strict left-to-right)

All other constructors evaluate arguments before their rule fires.

5.11 Static Well-Formedness

- KB items: `(: f T)`, `(= l r)`, `(=> l r)`, token binds, imports, and space operations are permitted.
- Patterns: variables appear only as `p-var` nodes.
- Spaces: every `match/add-atom/rem-atom/get-atoms/add-reduct` references a space reachable from the current program context.
- Tokens: token names are unique within a program file.
- Types: grounded objects may carry types; mismatch yields `BadType` or an `Error` atom.

5.12 Conformance

An IL-conformant engine must:

1. Accept the textual program format in Sec. 5.5.
2. Implement all edge families with the strictness contracts above.
3. Treat `Error` as a user-level symbol (no exceptions).
4. Preserve nondeterminism semantics (order is unspecified unless a deterministic profile is advertised).
5. Handle `=>` either as a directed rule or diagnose it clearly if unsupported.

5.13 Worked Micro-Examples (Textual IL)

All examples below follow the BNFC textual conventions (top-level `!` and `?`, final `.`).

Equality and call.

```
(: inc (-> Long Long))
(= (inc $x) (+ $x 1))

! (inc 2)
.
```

Match over a space.

```
(bind! &S (new-space))

! (add-atom &S (Parent Tom Bob))
? (match &S (Parent $x Bob) $x)
.
```

Sealed.

```
! (sealed ($x)
      (Cons $x (Cons $y Nil)))
.
```

Superpose and collapse.

```
(= (color) red)
(= (color) green)
(= (color) blue)

! (collapse (color))
.
```

Python interop with kwargs.

```
(bind! &np (py-atom "numpy"))
! ((py-dot &np arange)
    (kwargs (start 2) (stop 10) (step 3)))
.
```

5.14 From MeTTa Calculus to MeTTa-IL (one-step view)

We now briefly summarize how the mapping to MeTTa-IL plays with the MeTTa Calculus formulation.

Here spaces are names; facts/rules are guarded offers; COMM is a tiny transaction. The IL captures each such transaction with a small family of edges (Query/Chain/Transform/Add/Rem/Get/Match/Output) over explicit constructors. Concretely:

- **Equality at Eq.** A calculus offer $\text{for}((f \vec{p}) \leftrightarrow \text{Eq}) \text{emit}(\mathbf{W}, r)$ becomes an IL equation $(= (f \vec{p}) r)$. Firing corresponds to E Query/E Chain (Sec. 4.8.1), delivering $r\sigma$ to the workspace/output.
- **Space match Σ .** $\text{for}(p \leftrightarrow \Sigma) \text{emit}(\mathbf{W}, r)$ becomes $(\text{match } \Sigma \ p \ r)$ with E Match (Sec. 4.8.2).
- **Facts.** A persistent calculus fact $! \text{emit}(\Sigma, a)$ corresponds to $(\text{add-atom } \Sigma \ a)$; withdrawal to $(\text{rem-atom } \Sigma \ a)$ (E Add/E Remove).
- **Get/collapse.** Enumeration via E Get; collection via E Collapse (Sec. 4.8.4).
- **Freshness/sealed.** `new` and `capture?avoidance` compile to IL `sealed` (E Seal, Sec. 4.8.5) and, for spaces, to `new-space+binding` (Sec. 4.7.5, 3.6.6).
- **Output.** The calculus? "insensitive then re?emit to O" is IL's E Output (Sec. 4.8.5).
- **Grounders/interop.** Calculus ground processes G map to IL grounded calls (numeric/relational/I/O and Python) with the dedicated edge families (Sec. 4.8.6).

Equivalence intuition (bisimulation up to scheduling). Install (i) one calculus "driver" $\text{for}(_ \leftrightarrow \text{Eval})$ for each redex and (ii) offers at **Eq** and spaces. Every machine step in Sec. 3 corresponds to a finite sequence of COMMs that IL realizes by one edge firing; conversely, each IL edge firing mirrors a COMM between the corresponding offers. Thus observable results (modulo permutation of independent results) coincide.⁴

6 User-Customizable Evaluation Order in MeTTa

6.1 Overview and Motivation

This section proposes an extension to MeTTa that enables fine-grained, per-symbol control over evaluation order and strictness. While MeTTa's current model (strict for most arguments, non-strict for Atom-typed parameters) works well in practice, certain use cases benefit from more precise control:

- **Efficiency:** Implementing short-circuit evaluation without special forms
- **Laziness:** Supporting by-need evaluation with memoization for expensive computations
- **Parallelism:** Marking arguments that can be evaluated concurrently

⁴Machine small?step rules are in Sec. 3.4?3.6; IL edges in Sec. 4.8; the "spaces as tags" reading that makes COMM transactional is explicit in the calculus note (pp. 2?5).

⁵In the BNFC textual IL the head is `rem-atom` (not `remove-atom`); see the IL conventions.

- **Custom ordering:** Evaluating arguments in non-standard sequences for semantic or performance reasons

The proposal adds:

1. A declarative syntax for specifying evaluation policies per symbol
2. Minimal extensions to the operational semantics (thunks and forcing)
3. Corresponding IL support with backward compatibility

Key principle: This is an opt-in extension. Programs without evaluation policies behave exactly as before.

6.2 Surface Syntax

6.2.1 Evaluation Policy Declarations

We introduce a single declaration form:

```

EvalDecl ::= '(' 'eval-policy' Target EvalSpec ')'

Target    ::= Symbol                                ; the function
              being configured

EvalSpec ::= '(' 'args' { Mode } ')'                ; one Mode per
              positional arg
              [ '(' 'order' Order ')' ]              ; evaluation
              sequence
              [ '(' 'result' Mode ')' ]               ; result
              normalization
              [ '(' 'parallel' { Nat } ')' ]           ; concurrent
              evaluation hints
              [ '(' 'name' { Nat } ')' ]               ; shorthand
              for by-name args
              [ '(' 'need' { Nat } ')' ]               ; shorthand
              for by-need args

Mode      ::= 'val' | 'name' | 'need'
Order     ::= 'ltr' | 'rtl' | '(' 'custom' { Nat } ')'
```

6.2.2 Evaluation Modes

- **val** (by-value): Evaluate argument before the call (strict)
- **name** (by-name): Pass unreduced, never force at call site
- **need** (by-need): Pass as thunk, force on demand with memoization

6.2.3 Examples

```
; Document default 'if' behavior explicitly
(eval-policy if (args val name name) (order ltr))

; All-strict function with left-to-right evaluation
(eval-policy f+ (args val val val) (order ltr))

; Lazy map: function passed syntactically, list
; evaluated
(eval-policy map (args name val) (order ltr))

; Short-circuit 'and': both lazy, force as needed
(eval-policy my-and (args need need) (order ltr))

; Parallel evaluation hint: args 2 and 3 can run
; concurrently
(eval-policy g (args val val val)
               (order custom 2 3 1)
               (parallel 2 3))
```

6.3 Operational Semantics

6.3.1 Extended Machine State

We extend the register machine with:

- A policy table Π mapping symbols to their evaluation policies
- A thunk store μ for by-need memoization (conceptual; implementations may use graph annotation)

State becomes: $\langle I, K, W, O \mid \vartheta, \mathcal{M}, \nu, \Pi, \mu \rangle$

6.3.2 Policy Structure

For symbol f :

$\Pi(f) = (\text{modes} = \langle m_1, \dots, m_n \rangle, \text{order} = o, \text{parallel} = P \subseteq \{1..n\}, \text{result} = m_{res})$

6.3.3 Argument Preparation Phase

For application $f(t_1, \dots, t_n)$ with policy $\Pi(f)$:

1. **Schedule:** Determine evaluation order from policy
2. **Prepare:** For each argument position i in order o :

- If $m_i = \text{val}$: Evaluate $t_i \rightarrow^* v_i$ to normal form
 - If $m_i = \text{need}$: Wrap as $\text{delay}(t_i)$ (thunk)
 - If $m_i = \text{name}$: Leave t_i unreduced
3. **Parallelize**: Arguments in set P may be evaluated concurrently
 4. **Apply**: Use prepared arguments $\vec{t}^*$ in standard Query/Chain

6.3.4 Thunk Semantics (By-Need)

Delay and force operations.

- $\text{delay}(e)$ creates thunk with fresh label α , stores $\mu[\alpha] = \text{Uneval}(e)$
- First force: $\text{force}(\text{thunk } \alpha)$ evaluates $e \rightarrow^* v$, updates $\mu[\alpha] := \text{Val}(v)$, returns v
- Subsequent forces: Return memoized v from $\mu[\alpha]$

Memoization scope. Memoization is per-thunk-node and per-nondeterministic-branch. The same thunk forced twice in one branch returns the cached value. Different thunk nodes don't share results. When computation branches nondeterministically, each branch maintains its own memo table.

6.3.5 Result Policy

If $\Pi(f).\text{result} = \text{val}$, normalize the result $u \rightarrow^* v$ before enqueueing. Default is **name** (no normalization).

6.3.6 Policy Precedence and Special Cases

Precedence rules:

1. Head-specific **eval-policy** overrides type-based defaults
2. Last declaration wins within a module (tools should warn)
3. Special forms' non-strict contracts are normative unless explicitly overridden

Higher-order calls. Policies apply to the *runtime head* after reduction. No policy composition or merging occurs. The head symbol or grounded operator determines the policy.

Interaction with nondeterminism. When preparing arguments encounters nondeterminism (e.g., **superpose**), branches are explored at that point before the callee's rewrite. Result sets are unchanged (up to permutation), but observable effects may reflect the chosen order.

6.4 Design Principles

- **Local and explicit:** Policies are per-symbol, not global
- **Backward compatible:** No policy means current behavior
- **Sound with unification:**
 - **val** arguments unify against evaluated forms
 - **name** arguments unify against surface terms
 - **need** arguments unify against thunks (force when needed)
- **Parallelism as permission:** The **parallel** annotation authorizes interleaving; sequential execution remains compliant
- **Portability note:** Custom evaluation orders may expose non-confluence with side effects or space mutation

6.5 IL Extension (BNFC textual IL)

This section gives a BNFC-compatible, textual IL presentation of the evaluation-policy extension (user-configurable argument modes and evaluation order), together with two new heads **delay** and **force**. All examples are in the S-expression IL used by the reference BNFC parser (top-level items; final dot).

6.5.1 Top-level policy declarations

Policies are ordinary top-level items; engines that do not implement the extension **MUST** still parse them and **MAY** ignore them at runtime.

```
Item      ::= ...
            | (policy <head>
                (args    <mode> ...)
                (order   <ord>)
                [ (parallel <i> <j> ...) ]
                [ (result <mode>) ])

<mode>    ::= val | name | need
<ord>     ::= ltr | rtl | (custom <i> <j> ... <k>)
<head>    ::= a symbol naming the runtime head (e.g., my
              -and, pmap)
<i>,<j>    ::= 1-based argument positions; must be within
              arity
```

Meaning. **args** gives the per-parameter mode: **val** (strict, evaluate before call), **name** (pass unreduced), **need** (pass as thunk; force on demand). **order** selects left-to-right, right-to-left, or an explicit permutation. **parallel** lists argument positions that may be evaluated concurrently (subject to effects and fairness). **result** optionally states how to treat the result (**val** to normalize; **name** to leave as-is; **need** to return a thunk).

Precedence. If multiple policies are declared for the same head in one file, the last one wins. Special forms with normative non-strictness keep their behavior unless a tool explicitly opts in to override them.

6.5.2 New heads (textual IL)

The extension introduces two ordinary heads:

```
; Heads (no special syntax; ordinary S-expr calls)
(delay e)      ; construct a thunk for e (no evaluation)
(force x)      ; demand the value; memoize if x is a
                thunk
```

Operational intent. `(delay e)` is a value (no step). `(force (delay e))` forces `e` (under the current strategy), memoizes its value, and returns it; `(force v)` with non-thunk `v` is the identity.

6.5.3 Edge families (small-step)

- **E_Delay:** $(\text{delay } e)$ is a value; no reduction until forced.
- **E_ForceUneval:** $(\text{force } (\text{delay } e)) \rightarrow v$ after $e \Downarrow v$; memoize v .
- **E_ForceVal:** $(\text{force } v) \rightarrow v$ when v is not a thunk (idempotent).

6.5.4 Application elaboration (textual IL)

Given a call `(h a1 ... an)` in textual IL:

1. Evaluate the head to a runtime head `f` (symbol or grounded).
2. Look up policy $\Pi(f)$. If none, use legacy behavior (engine default).
3. Transform each argument by its mode:
 - **val:** leave as-is (it will be evaluated per scheduling).
 - **name:** pass the unreduced term.
 - **need:** wrap as `(delay ai)`.
4. Schedule evaluation of **val** arguments per **order** and **parallel**. **name** and **need** are not evaluated here.
5. Form the call `(f a1* ... an*)` using the transformed args.
6. If `(result val)` is declared, normalize the result value before enqueueing it; otherwise return it as produced.

For **need**, the body of `f` must *force* thunks at the points where their values are required. Memoization scope follows Section on by-need (per thunk node, per nondet branch).

6.5.5 Lowering from surface to textual IL

Surface `(f a1 ... an)` already has textual IL shape. The lowering pass:

1. Lowers `f` and `ai` subtrees recursively.
2. Resolves $\Pi(f)$ from the nearest policy declaration in scope.
3. Rewrites each `ai` to `ai*` by mode (`need` adds `(delay ...)`).
4. Inserts administrative `let` or `progn` if needed to enforce `order` and `parallel` on strict arguments.
5. Emits the call `(f a1* ... an*)`.
6. Inserts `force` at strict use sites inside the definition(s) of `f` where evaluation of `need` arguments is semantically required.

6.5.6 Well-formedness

- The arity of `(args ...)` must match the callable arity.
- Positions in `(order custom ...)` and `(parallel ...)` must be in $1..n$ and pairwise distinct; positions named in `parallel` are evaluated concurrently.
- Engines MUST treat special forms' non-strict positions as normative unless explicitly configured to override.
- Parsers MUST accept `(policy ...)` even if the runtime ignores policies.

6.6 Examples (BNFC textual IL)

All examples below are complete textual IL files; the final dot terminates the file.

6.6.1 Short-circuit boolean with on-demand forcing

```
(policy my-and (args val need) (order ltr))

(= (my-and true  $rhs) (force $rhs))
(= (my-and false $rhs) false)

! (my-and true  (println! "forcing rhs") )
! (my-and false (println! "not forced") )
.
```

The first call prints and returns the value of `(println! ...)`, the second does not print because the `need` argument is never forced.

6.6.2 By-need memoization

```
(policy costly (args need) (order ltr))

(= (twice-costly $x)
   (let $y (costly $x)
       (+ (force $y) (force $y))))

! (twice-costly 41)
.
```

(costly \$x) is computed once; both forces read the memoized result in the same nondeterministic branch.

6.6.3 Parallel evaluation of a strict argument

```
(policy pmap (args name val) (parallel 2) (order ltr))

(= (pmap $f $xs)
   (case $xs
       ((()) ())
       (((Cons $h $t)) (Cons ($f $h) (pmap $f $t)))))

! (pmap (lambda1 $z (+ $z 1)) (Cons 1 (Cons 2 (Cons 3
  ())))))
.
```

Argument 2 (the list) may be evaluated concurrently with the head call to \$f on \$h. Engines remain responsible for effect ordering: prints and space updates reflect the chosen schedule.

6.7 Summary

This extension provides:

- **Syntax:** (eval-policy f (args ...) (order ...) ...)
- **Semantics:** Policy table + argument preparation + thunk/force
- **IL:** Policy metadata + delay/force nodes + elaboration rules
- **Compatibility:** Full backward compatibility when unused

The design keeps MeTTa's core model (equations + unification + rewriting) while giving advanced users precise control over evaluation strategy where needed.

7 A Few Concrete Examples

In this section we work through concrete MeTTa code examples, demonstrating how they instantiate the formal syntax and reduce under the operational semantics given above. These examples validate that our formalism correctly captures actual MeTTa programming patterns.

7.1 Syntax and Semantics of Simple MeTTa Code Examples

We explain two MeTTa code snippets by systematically analyzing their syntax against the formal grammar, their reduction under the operational semantics, and their translation to IL.

7.1.1 EXAMPLE A — recursive mapper over expressions

Source

```
(: map-expr (-> (-> $t $t) Expression Expression))
(= (map-expr $f $expr)
  (if (== $expr ()) ()
      (let* (($head (car-atom $expr))
             ($tail (cdr-atom $expr))
             ($head-new ($f $head))
             ($tail-new (map-expr $f $tail)))
        (cons-atom $head-new $tail-new))))
! (map-expr not (False True False False))
```

Syntax (surface grammar) — parsing and classification Each line parses according to our formal grammar (Section 2):

Type declaration: `(: map-expr (-> (-> $t $t) Expression Expression))`

- Matches production `TypeDecl ::= '(' ':' Pattern TypeExpr ')'`
- The type `(-> (-> $t $t) Expression Expression)` follows the right-associative arrow rule
- `$t` is a type variable (polymorphic parameter)
- `Expression` is a built-in meta-type for S-expressions

Definitional equality: `(= (map-expr $f $expr) BODY)`

- Matches `DefEq ::= '(' '=' Pattern Form ')'`
- Left side is a pattern with head `map-expr` and variable patterns `$f`, `$expr`
- This adds an equality to the knowledge base `K` for rewriting

Control structures in the body:

The `if` construct:

- Has shape `(if <Bool-expr> <then-Atom> <else-Atom>)`
- Per the earlier strictness table: position 1 is strict (guard evaluates), positions 2-3 are non-strict (branches passed as Atoms)
- The guard `(== $expr ())` uses the grounded equality operator

The `let*` construct:

- Sequential bindings where each RHS evaluates before binding
- Each binding has shape `(<Pattern> <Form>)` where the Form is strict
- Variables bound in earlier clauses are visible in later ones

Expression manipulation operators:

- `car-atom`, `cdr-atom`, `cons-atom` are grounded operations on meta-type Expression
- They operate at the syntactic level on S-expressions
- Arguments of type Expression are passed as Atoms (non-strict) per the library convention

Dynamics (small-step operational semantics) Let's trace the evaluation of `! (map-expr not (False True False False))` using the formal semantics from Section 3:

Step 1: Query rule application

- The machine state has the call in input register I
- Query rule unifies with `(= (map-expr $f $expr) ...)`
- Unification yields: $\sigma = \{ \$f \mapsto \text{not}, \$expr \mapsto (\text{False True False False}) \}$
- The instantiated body moves to workspace W

Step 2: Evaluate the if guard

- Must evaluate `(== (False True False False) ())` per if's strictness
- The grounded `==` operator compares structural equality
- Result: `False` (non-empty list ? empty list)
- By `E_IfFalse` rule: the else-branch Atom is selected for evaluation

Step 3: Process `let*` bindings The `let*` desugars to nested lets. Each binding evaluates as follows:

1. `$head := (car-atom (False True False False))`
 - Grounded operation extracts first element
 - Result: `False`
2. `$tail := (cdr-atom (False True False False))`
 - Grounded operation removes first element
 - Result: `(True False False)`
3. `$head-new := (not False)`
 - Apply the function `not` (passed as `$f`)
 - Grounded boolean operation yields `True`
4. `$tail-new := (map-expr not (True False False))`
 - Recursive call triggers Query rule again
 - By identical reduction steps, yields `(False True True)`

Step 4: Construct result

- `(cons-atom True (False True True))` builds the final expression
- Result moves through `W` to output `O`: `(True False True True)`

This evaluation sequence precisely follows the Meta-MeTTa machine model: Query/Chain for equalities, strict evaluation contexts per the table, and grounded operations for built-ins.

IL translation The transpiler produces this MeTTa-IL, preserving strictness annotations:

```
(: map-expr (-> (-> $t $t) Expression Expression))
(= (map-expr $f $expr)
  (if (== $expr ())
    ()
    (let* (($head (car-atom $expr))
          ($tail (cdr-atom $expr))
          ($head-new ($f $head))
          ($tail-new (map-expr $f $tail)))
      (cons-atom $head-new $tail-new))))
! (map-expr not (False True False False))
.
```

Key IL mappings:

- `if` node explicitly marks guard as strict, branches as Atoms

- `let*` preserves sequential evaluation order
- `call` distinguishes grounded operations from `app` (general application)
- Variables use `(var ...)` nodes, symbols use `(sym ...)`

7.1.2 EXAMPLE B — Python interop, grounded atoms, keyword args

Source

```
! (bind! np (py-atom numpy))
! (bind! np-arange (py-atom numpy.arange))
! (np-arange (Kwargs (start 2) (stop 10) (step 3)))

! ((py-dot np abs) ((py-dot np random.randint) -25 0))

! (bind! abs (py-atom numpy.absolute (-> Number Number)))
! (+ (abs -5) 10)
```

Syntax — token binding and grounded operations Symbol binding mechanism: `bind!`

- Matches `SymbolBind ::= '(' 'bind!' Symbol Form ')'`
- Evaluates the `Form` and binds the resulting atom to `Symbol` in the current runtime environment
- No parse-time substitution and no token map; references to `Symbol` are ordinary symbol lookups at evaluation time

Python interop constructs:

`py-atom` (grounded import):

- Type: `(-> String [TypeExpr] Grounded)`
- Evaluates Python expression/import, returns grounded object
- Optional type annotation enables dynamic checking
- The grounded object wraps the Python value for MeTTa use

`py-dot` (attribute access):

- Type: `(-> Grounded Symbol Grounded)`
- Implements Python's dot notation for attribute/method access
- Can chain: `(py-dot np random.randint)` means `np.random.randint`

`Kwargs` (keyword arguments):

- Bundles keyword arguments for Python functions
- Shape: `(Kwargs (<Symbol> <Form>) ...)`
- Each pair becomes a keyword argument in the Python call

Dynamics — evaluation with token expansion and grounded calls
Token registration phase:

```
! (bind! np (py-atom numpy)):
```

1. Evaluate (py-atom numpy) using E_PyAtom rule
2. Python import returns grounded module object G(numpy)
3. Install binding: $\sigma[\text{np}] := G(\text{numpy})$
4. Future parsing replaces np with G(numpy)

```
! (bind! np-arange (py-atom numpy.arange)):
```

1. py-atom evaluates the Python expression numpy.arange
2. Returns grounded callable G(numpy.arange)
3. Install binding: $\sigma[\text{np-arange}] := G(\text{numpy.arange})$

Grounded function call with kwargs:

```
! (np-arange (Kwargs (start 2) (stop 10) (step 3))):
```

1. Parser has already substituted: np-arange $\rightarrow G(\text{numpy.arange})$
2. The call shape is (G(numpy.arange) (Kwargs ...))
3. E_GCall rule applies: invoke Python function with keyword arguments
4. Python executes: numpy.arange(start=2, stop=10, step=3)
5. Returns grounded NumPy array object

Chained attribute access and calls:

```
! ((py-dot np abs) ((py-dot np random.randint) -25 0)):
```

1. Inner: (py-dot np random.randint)
 - np already replaced with G(numpy)
 - E_PyDot: access random attribute, then randint
 - Result: G(numpy.random.randint)
2. Inner call: (G(numpy.random.randint) -25 0)
 - E_GCall with positional arguments
 - Returns random integer in [-25, 0)
3. Outer: (py-dot np abs) yields G(numpy.abs)
4. Outer call applies numpy.abs to the random integer

Typed grounded binding:

```
! (bind! abs (py-atom numpy.absolute (-> Number Number))):
```

- The type annotation attaches to the grounded object
- Future calls to abs check argument/return types
- Type mismatches yield BadType atoms (not exceptions)

IL translation Token bindings become module header declarations:

```
! (bind! np (py-atom "numpy"))
! (bind! np-arange (py-atom "numpy.arange"))

! (np-arange (Kwargs (start 2) (stop 10) (step 3)))

! ((py-dot np abs) ((py-dot np random.randint) -25 0))

! (bind! abs (py-atom "numpy.absolute" (-> Number Number)))
! (+ (abs -5) 10)
.
```

IL translation strategy:

- `bind!` effects captured as static (`token ...`) declarations
- `gconst` nodes represent grounded constructors
- `gcall` distinguishes grounded calls from symbolic `call`
- Keywords preserved as structured (`kwargs ...`) nodes

7.1.3 Why these explanations/ILs line up with the formal semantics

These examples demonstrate key alignments between practice and formalism:

Evaluation contexts: The IL preserves exactly the strictness positions from the strictness table. The `if` guard evaluates while branches remain Atoms until selected—this matches both the operational semantics (`E_IfTrue/E_IfFalse`) and standard library usage.

Sequential evaluation: The `let*` translation to nested `lets` ensures left-to-right binding evaluation, matching the given semantics where each RHS reduces before binding.

Runtime binding only: `bind!` installs a symbol-to-atom binding in the current runtime environment; there is no token-expansion phase.

Grounded operations: Expression constructors and Python interop follow the grounded operation semantics: strict evaluation of arguments (except those typed Atom), then host-level execution, returning grounded results.

Register machine alignment: The execution traces follow the Meta-MeTTa four-register model: equalities trigger Query/Chain rules moving terms through $I \rightarrow K \rightarrow W \rightarrow O$, with grounded operations as Transform steps.

7.2 Example: The Red and Black Lambda Calculus in MeTTa

This example explores a fundamental challenge in MeTTa: implementing parametric algebraic data types (ADTs) with proper namespacing. We examine

several approaches to encoding the lambda calculus with colored terms, demonstrating how MeTTa’s equation-based semantics and nondeterminism interact with type declarations.

7.2.1 Code Snippets in Question

ADT’s are classically represented in MeTTa as follows:

```
; type M ::= B | V | lambda V.M | (M M)
(: Term Type)
(: Ground (-> B Term))
(: Mention (-> V Term))
(: Abstraction (-> V (-> Term Term)))
(: Application (-> Term (-> Term Term)))
```

For coloring the Lambda Calculus, we need to parametrize the above ADT in B and V. The naive way to do this is wrapping the subspace in a function:

```
(= (LambdaTheory $B $V) (superpose (
(: Term Type)
(: Ground (-> $B Term))
(: Mention (-> $V Term))
(: Abstraction (-> $V (-> Term Term)))
(: Application (-> Term (-> Term Term)))
)))
```

In both cases, the terms are not contained in an encompassing theory: `(: Application (-> Term (-> Term Term)))` does not know what parameters went into the function. Computationally: if you can’t tell the difference, does it matter? In most languages, there’s more to a type than just its inner name. In Scala, there’s a qualified path to that name, see below for the example: `root.RedBlackLambda.RC.Term` and `root.RedBlackLambda.BC.Term` are different types. We can wrap a result in a container, effectively resulting in a path: `(RedLambda (LambdaTheory BTheory String))` results in

```
(RedLambda (: Term Type))
(RedLambda (: Ground (-> BTheory Term)))
(RedLambda (: Mention (-> String Term)))
(RedLambda (: Abstraction (-> String (-> Term Term))))
(RedLambda (: Application (-> Term (-> Term Term))))
(RedLambda (= ((equivalent $l) $r) ???)) ; added to make the result more tangible
```

It doesn’t seem unreasonable to have functions wrapped in an object, especially if you consider prefix compression. However, we get something different than we meant: we want to transport the types to the global namespace, but make them unique. We could solve this by parametrization, take:

```

(= (LambdaTheory $F $B $V) (superpose (
  (: ($F Term) Type)
  (: ($F Ground) (-> $B ($F Term)))
  (: ($F Mention) (-> $V ($F Term)))
  (: ($F Abstraction) (-> $V (-> ($F Term) ($F Term))))
  (: ($F Application) (-> ($F Term) (-> ($F Term) ($F Term))))
)))

```

(LambdaTheoryF RC BTheory String) now results in

```

(: (RC Term) Type)
(: (RC Ground) (-> BTheory (RC Term)))
(: (RC Mention) (-> String (RC Term)))
(: (RC Abstraction) (-> String (-> (RC Term) (RC Term))))
(: (RC Application) (-> (RC Term) (-> (RC Term) (RC Term))))

```

7.2.2 What each fragment is syntactically

A. The unparametrized ADT (algebraic signature)

```

; type M ::= B | V | lambda V.M | (M M)
(: Term Type)
(: Ground      (-> B Term))
(: Mention     (-> V Term))
(: Abstraction (-> V (-> Term Term)))
(: Application (-> Term (-> Term Term)))

```

Syntactic analysis: Each line follows the type declaration production given above:

- TypeDecl ::= '(' ' ':' Pattern TypeExpr ')'
- Term becomes a type constant via (: Term Type)
- Constructor signatures use right-associative arrows: (-> V (-> Term Term)) parses as $V \rightarrow (Term \rightarrow Term)$

Operational behavior: Under the semantics given above, type declarations populate the type environment Δ but do not reduce. They act as static constraints: when **Ground** is later used in an equality like $(= (\text{Ground } b) \dots)$, the type checker verifies that $b : B$ and the result has type **Term**.

The limitation: This encoding has no parameters—**B** and **V** are global type variables. Different instantiations would collide in the same namespace.

B. The naive parameterization (returning a bundle of facts)

```

(= (LambdaTheory $B $V) (superpose (
  (: Term Type)
  (: Ground      (-> $B Term))

```

```

(: Mention      (-> $V Term))
(: Abstraction (-> $V (-> Term Term)))
(: Application (-> Term (-> Term Term)))
)))

```

Syntactic structure: This creates a function via definitional equality:

- `DefEq ::= '( ' '=' Pattern Form ' )'`
- LHS pattern: `(LambdaTheory $B $V)` with variables for parameters
- RHS: `superpose` applied to a tuple of type declarations

Operational semantics: When `(LambdaTheory T1 T2)` is called:

1. Query rule (Section 3.3) unifies with the LHS pattern
2. Substitution: $\$B \rightarrow T1$, $\$V \rightarrow T2$ in the RHS
3. `superpose` rule: tuple elements become separate results
4. Each `(: ...)` declaration moves through `W` to `O`

Result: Five type declarations with `T1` and `T2` substituted for parameters.

The problem: While parameterized, the resulting `Term` type is still unqualified. Two calls like `(LambdaTheory Bool String)` and `(LambdaTheory Int Char)` both produce a `Term` type—they collide in the global namespace. The parameters are lost after instantiation.

C. Namespacing by wrapping (syntactic container)

```
(RedLambda (LambdaTheory BTheory String))
```

Syntactic analysis: Pure function application without a reduction rule:

- `RedLambda` has no associated equality in `K`
- Application remains unreduced as data structure
- Each of the five results from `superpose` gets wrapped

Operational behavior:

1. Inner `(LambdaTheory BTheory String)` reduces to five results via `superpose`
2. Each result `r` becomes `(RedLambda r)`
3. No further reduction occurs (no equality for `RedLambda`)
4. Results are structural terms: `(RedLambda (: Term Type))`, etc.

Why this partially works: The wrapper provides a namespace prefix. Pattern matching can distinguish `(RedLambda (: Term Type))` from `(BlackLambda (: Term Type))`.

Why it's incomplete: The wrapped declarations aren't usable as regular types. You can't write `(: myFunc (-> (RedLambda Term) Bool))` because `(RedLambda Term)` isn't recognized as a type—it's just an unreduced expression.

D. Qualifying the names themselves (parametric functor)

```
(= (LambdaTheory $F $B $V) (superpose (  
  (: ($F Term) Type)  
  (: ($F Ground)      (-> $B ($F Term)))  
  (: ($F Mention)     (-> $V ($F Term)))  
  (: ($F Abstraction) (-> $V (-> ($F Term) ($F Term))))  
  (: ($F Application) (-> ($F Term) (-> ($F Term) ($F Term))))  
)))
```

Key syntactic innovation: Using `($F Term)` creates a compound symbol:

- In surface syntax, this is just an S-expression `(RC Term)`
- The type system treats it as a qualified name
- `($F Term)` in type position becomes a distinct type for each `$F`

Operational semantics: Calling `(LambdaTheory RC BTheory String)`:

1. Query unifies: $\$F \rightarrow RC$, $\$B \rightarrow BTheory$, $\$V \rightarrow String$
2. Substitution produces `(: (RC Term) Type)`, etc.
3. Each declaration has qualified names: `(RC Term)`, `(RC Ground)`
4. These are distinct from `(BC Term)`, `(BC Ground)`

Why this succeeds: The qualified symbols are first-class types in the global namespace. You can write `(: myFunc (-> (RC Term) (BC Term)))` and the type system recognizes both as valid, distinct types.

7.2.3 How the semantics justifies what happens

The core evaluation mechanism: All approaches use the same Meta-MeTTa register machine:

1. Function call enters input register I
2. Query rule matches against equalities in K
3. Unification binds pattern variables
4. Instantiated RHS moves to workspace W
5. Results propagate to output O

Role of nondeterminism: The `superpose` operation is crucial:

- Transforms a tuple into multiple alternative results
- Each alternative is processed independently

- No reduction occurs inside type declarations—they're values
- Enables "returning multiple declarations" from one function

Type declaration semantics: Key insight: $(: \text{ s T})$ declarations:

- Add typing facts to environment Δ
- Do not themselves reduce
- Act as constraints for later term construction
- Can contain compound expressions as type names

Why qualification works: The expression (RC Term) serves dual roles:

1. **Syntactically:** It's a valid S-expression by the grammar
2. **Type-theoretically:** It's treated as an atomic type name
3. **Operationally:** No reduction rule applies (no equality for RC as function)
4. **Semantically:** Type checker recognizes it as a distinct type constant

7.2.4 A compact MeTTa-IL that a transpiler can emit

The IL design explicitly supports these patterns through careful representation choices:

IL Core (schema)

```

; Modules & declarations
(MODULE <sym> ...)                ; optional grouping
(TYPE <sym>)                      ; declare a type constant
(SIG <sym> <type>)                ; declare a typed operator

; Types (right-assoc)
(TY <sym>)                       ; type constant/name
(ARROW <type> <type>)             ; function arrow

; Functions (rules)
(DEF <head-pattern> <body>)       ; (= <head> <body>) at IL-level

; Values, variables, apps
(SYM <name>)                     ; a bare symbol
(VAR <name>)                     ; a pattern variable $name
(APP <f> <arg1> ... <argn>)       ; application

; Non-deterministic bundles
```



```

(SUPERPOSE <list-of-values>)          ; explicit non-deterministic result set
(TUPLE <v1> ... <vn>)                  ; convenience carrier for lists

; Qualified symbols (namespacing-as-construction)
(QUAL <q> <sym>)                        ; ($q sym) -- a new symbol

```

Design rationale:

- **SUPERPOSE:** Direct representation of nondeterministic branching
- **QUAL:** Preserves compound names without premature string mangling
- **TYPE/SIG:** Static declarations matching MeTTa's type facts
- **DEF:** Equality rules drive the rewrite system

IL for the unparametrized ADT

```

; LambdaCore
(: Term Type)
(: Ground      (-> B Term))
(: Mention     (-> V Term))
(: Abstraction (-> V (-> Term Term)))
(: Application (-> Term (-> Term Term)))
.

```

Direct translation: each `(:...)` becomes a **TYPE** or **SIG** declaration.

IL for the naive LambdaTheory \$B \$V

```

(= (LambdaTheory $B $V)
  (superpose (
    (: Term Type)
    (: Ground      (-> $B Term))
    (: Mention     (-> $V Term))
    (: Abstraction (-> $V (-> Term Term)))
    (: Application (-> Term (-> Term Term)))
  )))
.

```

The equality becomes a **DEF** rule; `superpose` maps to **SUPERPOSE**.

IL for the “wrapped” variant (RedLambda (LambdaTheory ...))

```

(RedLambda (LambdaTheory BTheory String))
.

```

Pure structural term—no reduction rule, just data.

IL for the qualified-name version LambdaTheory \$F \$B \$V

```
(= (LambdaTheory $F $B $V)
  (superpose (
    (: ($F Term) Type)
    (: ($F Ground)      (-> $B ($F Term)))
    (: ($F Mention)     (-> $V ($F Term)))
    (: ($F Abstraction) (-> $V (-> ($F Term) ($F Term))))
    (: ($F Application) (-> ($F Term) (-> ($F Term) ($F Term))))
  )))
.
```

The QUAL node: Represents qualified names without string concatenation. A backend can later decide whether (QUAL RC Term) becomes RC.Term, RC::Term, or remains structural.

Example instantiation. The surface call

```
(LambdaTheory RC BTheory String)
.
```

IL-reduces to the bundle

```
(superpose (
  (: (RC Term) Type)
  (: (RC Ground)      (-> BTheory (RC Term)))
  (: (RC Mention)     (-> String  (RC Term)))
  (: (RC Abstraction) (-> String  (-> (RC Term) (RC Term))))
  (: (RC Application) (-> (RC Term) (-> (RC Term) (RC Term))))
))
.
```

A materializer pass can finalize names, e.g. (QUAL RC Term) to RC.Term, and emit the concrete environment:

```
TYPE RC.Term
SIG  RC.Ground      : BTheory -> RC.Term
SIG  RC.Mention     : String  -> RC.Term
SIG  RC.Abstraction : String  -> RC.Term -> RC.Term
SIG  RC.Application : RC.Term -> RC.Term -> RC.Term
```

7.2.5 Why this IL is faithful and useful to your transpiler

Evaluation fidelity: The IL execution model precisely mirrors Meta-MeTTa:

- DEF rules → equality-based rewriting via Query/Chain
- SUPERPOSE → nondeterministic branching in workspace
- TYPE/SIG → static type environment population

- Register flow: I (input) $\rightarrow$ K (match) $\rightarrow$ W (work) $\rightarrow$ O (output)

Namespace preservation: The QUAL node maintains structure without premature commitment:

- Type checking sees (QUAL RC Term) as distinct from (QUAL BC Term)
- No string mangling during compilation
- Backend chooses final representation (structural or string-based)

Compositional correctness: Each language feature maps to exactly one IL construct:

- Surface $= \rightarrow$ IL DEF
- Surface **superpose** $\rightarrow$ IL SUPERPOSE
- Surface (\$F X) $\rightarrow$ IL (QUAL (VAR F) (SYM X))
- Surface : $\rightarrow$ IL TYPE or SIG

Tool-friendly design:

- SSA-like structure (no mutation)
- Explicit variable/symbol distinction
- Right-associative arrows made explicit
- Direct correspondence to register machine states

7.2.6 Optional: how a loader would use these results

If you want LambdaTheory to populate a space (rather than just return results), a tiny conventional wrapper suffices:

```
(= (install-LambdaTheory $F $B $V $space)
  (match &self
    (LambdaTheory $F $B $V)
    (add-atom $space $x)))
```

Operational breakdown:

1. **match** queries **self** for results of (LambdaTheory \$F \$B \$V)
2. Each result binds to **\$x**
3. **add-atom** inserts the unreduced declaration into **\$space**
4. Process repeats for all five declarations

This leverages two key features:

- **match** arguments are Atoms (non-strict)–patterns aren’t pre-evaluated
- **add-atom** inserts its second argument unreduced–declarations remain as data

Summary

The progression from naive to qualified parameterization demonstrates MeTTa’s expressiveness:

- Type declarations are first-class values that can be computed
- Nondeterminism via **superpose** enables "returning multiple declarations"
- Compound expressions as type names provide namespacing
- The operational semantics treats all these uniformly

The IL faithfully preserves these semantics while providing a clean compilation target that supports both structural and name-based backends.

7.3 Example: Chess Queens Problem in MeTTa

This example demonstrates how a sophisticated algorithm—the N-queens backtracking search—is expressed in MeTTa using equational programming, higher-order functions, and pattern matching. We systematically analyze Mike Archbold’s implementation against our formal syntax and semantics, showing how complex control flow emerges from simple rewrite rules.

7.3.1 The snippet (for reference)

```
(= (empty-board) Nil)

(= (adjoin-position $new-row $k $rest-of-queens)
    (append $rest-of-queens (Cons (list ($k $new-row)) Nil)))

(= (select-k $k $positions)
    (if (== $k 1)
        (car-list $positions)
        (select-k (- $k 1) (cdr-list $positions))))

(= (row-safe? $k-col $k-row $positions)
    (null-list? (filter
        (lambda1 $x (and (== (cdr-list $x) (list ($k-row))) (not (== (car-list $x) $positions)))))

(= (col-safe? $k-row $positions)
    (let $iter (lambda3 $cur $pos $self
        (if (null-list? $pos)
            True
```

```

      (if (or (== (cdar-list $pos) (Cons (- $k-row $cur) Nil))
              (== (cdar-list $pos) (Cons (+ $k-row $cur) Nil)))
          False
          ($self (+ $cur 1) (cdr-list $pos) $self)))
    ($iter 1 (cdr-list (reverse $positions)) $iter)))

(= (safe? $k $positions)
   (let (Cons $k-col (Cons $k-row Nil)) (select-k $k $positions)
       (and (col-safe? $k-row $positions) (row-safe? $k-col $k-row $positions))))

(= (queens $board-size)
   (let $queen-cols (lambda2 $k $self
                         (if (= $k 0)
                             (list ((empty-board)))
                             (filter
                                (lambda1 $positions (safe? $k $positions))
                                (flatmap
                                   (lambda1 $rest-of-queens
                                         (map (lambda1 $new-row
                                                (adjoin-position $new-row $k $rest-of-queens))
                                                (enumerate-interval 1 $board-size)))
                                   ($self (- $k 1) $self))))))
     ($queen-cols $board-size $queen-cols)))

!(assertEqual
  (queens 4)
  (tree (((1 2) (2 4) (3 1) (4 3)) ((1 3) (2 1) (3 4) (4 2)))))

```

7.3.2 How each form fits the surface grammar

We systematically map each construct to our formal grammar from Section 2, demonstrating that even complex algorithms use only MeTTa's core features.

Definitional equalities as rewrite rules Each function definition follows the pattern:

```
(= (function-name pattern1 pattern2 ...) body)
```

This matches the grammar production:

- `DefEq ::= '(' '=' Pattern Form ')'`
- The LHS is a pattern with head symbol and variable patterns
- The RHS is an arbitrary Form (expression)

Operational impact: Each equality adds a rewrite rule to the knowledge base K. When a matching call appears, the Query rule unifies the call with the LHS pattern and replaces it with the instantiated RHS.

List data structures The code uses three list representations:

- **Cons lists:** `(Cons head tail)` with `Nil` as empty
- **Tuple construction:** `(list e1 e2 ...)` for fixed-size tuples
- **Hybrid:** Lists of tuples, e.g., `(Cons (list 1 2) Nil)`

Grammar classification: These are ordinary function applications where `Cons`, `list`, and `Nil` are symbols. The operational semantics treats them as constructor applications—they evaluate their arguments but don’t reduce further unless pattern-matched.

Control flow through special forms **The if construct:**

- Surface shape: `(if condition then-branch else-branch)`
- Strictness: Position 1 is strict, positions 2-3 are Atoms
- Operational rule: Evaluate condition to `Bool`, then select branch

This evaluation order is crucial: branches remain unevaluated until selected, enabling recursion without infinite loops.

The let construct:

- Two forms: binding variables and pattern matching
- `(let $var expr body)`: Evaluate `expr`, bind to `$var`, evaluate `body`
- `(let pattern expr body)`: Evaluate `expr`, unify with pattern, evaluate `body`

Pattern `let` enables destructuring: `(let (Cons $h $t) lst ...)` extracts head and tail.

Higher-order programming with lambdas MeTTa provides arity-specific lambda constructors:

- `lambda1`: Single argument closure
- `lambda2`: Two argument closure
- `lambda3`: Three argument closure

The self-passing pattern: Since MeTTa lambdas can’t directly reference themselves, recursive closures use an explicit self parameter:

```
(let $f (lambda2 $x $self ... ($self ...))
  ($f initial-value $f))
```

This “tying the knot” pattern appears in both `col-safe?` and `queens`.

Grounded operations Arithmetic (+, -), comparison (==), and logic (and, or, not) are grounded operations:

- Type signatures enforce argument types (e.g., `(-> Number Number Number)`)
- Arguments evaluate before the operation (strict evaluation)
- Results are deterministic ground values

7.3.3 Intended data representation

The algorithm represents a board configuration as a list of queen positions, ordered by column:

```
positions :: List(Position)
Position  = (column row) -- a 2-tuple using (list col row)
Board     = List of positions for columns 1..k
```

Example: For a 4x4 board with queens at (1,2), (2,4), (3,1), (4,3):

```
(Cons (list 1 2)
      (Cons (list 2 4)
            (Cons (list 3 1)
                  (Cons (list 4 3) Nil))))
```

Key invariant: The k-th element represents the queen in column k, so we only need to track row positions to avoid conflicts.

7.3.4 Operational semantics walk-through

We trace execution using the Meta-MeTTa four-register machine model, showing how high-level control flow emerges from low-level rewrite steps.

Base operations `empty-board`:

- Equality: `(= (empty-board) Nil)`
- When called: Query rule finds this equality, returns Nil
- Represents the empty board with no queens placed

`adjoin-position`:

- Purpose: Add a new queen at position (k, new-row) to existing board
- Implementation: `(append rest (Cons (list k new-row) Nil))`
- Operational steps:
 1. Create position tuple: `(list k new-row)`
 2. Wrap in singleton list: `(Cons ... Nil)`

3. Append to existing positions

select-k:

- Purpose: Extract the k-th position from the list
- Algorithm: Recursive list traversal with counter
- Base case: `k = 1` returns `(car-list positions)`
- Recursive: Decrement `k`, advance to tail via `(cdr-list positions)`

Safety predicates The safety checks implement the queens constraint: no two queens can attack each other.

Row safety (row-safe?):

Algorithm logic:

1. Filter positions to find those in the same row as `k-row`
2. Check if any have different column than `k-col`
3. If filter returns empty list, position is safe

Operational breakdown of the filter predicate:

```
(lambda1 $x
  (and (== (cdr-list $x) (list ($k-row))) ; same row?
        (not (== (car-list $x) $k-col))) ; different column?)
```

For position `$x = (list c r)`:

- `(car-list $x)` extracts column `c`
- `(cdr-list $x)` extracts `(list r)` (singleton list)
- Compare with `(list k-row)` to check row equality

Diagonal safety (col-safe?):

This is the most complex predicate, checking diagonal attacks:

1. Reverse the position list (newest queen last)
2. Start from the tail (oldest queens)
3. For each older queen at distance `cur`:
 - Check if row difference equals `cur` (diagonal attack)
 - If yes, return False
 - Otherwise, increment `cur` and check next older queen

The `lambda3` with self-passing implements this iteration:

- `$cur`: Distance from current queen being placed
- `$pos`: Remaining positions to check
- `$self`: The lambda itself for recursion

Combined safety (safe?):

Uses pattern-matching `let` to destructure the `k`-th position:

```
(let (Cons $k-col (Cons $k-row Nil)) (select-k $k $positions) ...)
```

Operational steps:

1. Evaluate `(select-k k positions)` to get `(list col row)`
2. Unify with pattern `(Cons $k-col (Cons $k-row Nil))`
3. If unification succeeds: bind `$k-col` and `$k-row`
4. Evaluate body: `(and (col-safe? ...) (row-safe? ...))`

The main search procedure `queens` implements backtracking search through functional composition:

```
(queens board-size) =
  let queen-cols = (lambda2 k self ...)
  in (queen-cols board-size queen-cols)
```

The recursive strategy:

1. **Base case (k=0):** Return list containing empty board
2. **Recursive case:**
 - Get all valid `(k-1)`-queen configurations recursively
 - For each configuration:
 - Try placing queen `k` in each row `1..board-size`
 - Keep only safe placements

Functional composition breakdown:

```
(filter safe?                                ; 3. Keep only safe boards
  (flatmap                                   ; 2. For each (k-1) board
    (lambda1 $rest
      (map                                   ; 1. Try each row
        (lambda1 $row (adjoin ...))
        (enumerate-interval 1 n)))
    (self (- k 1) self)))                   ; Recursive call
```

The composition reads bottom-up:

1. Recursively get `(k-1)`-queen boards
2. Expand each to `n` candidates (one per row)
3. Filter to keep only safe placements

Execution trace for (queens 4) Initial call triggers the equality for queens:

1. Bind `$board-size` $\rightarrow$ 4
2. Create closure `queen-cols`
3. Call `(queen-cols 4 queen-cols)`

At `k=4`:

- Recursive call: `(self 3 self)` gets all 3-queen boards
- For each 3-queen board, try rows 1-4 for column 4
- Filter with `(safe? 4 ...)` keeps valid 4-queen boards

This continues recursively until `k=0` returns `(list (Nil))`, then builds up complete solutions.

7.3.5 Translation to MeTTa-IL (the transpiler's core IR)

The IL translation preserves the operational semantics while making evaluation order explicit.

IL summary (excerpt; same as in previous messages):

- `Def{name, params, body}`
- `App(fn, [args...])` function/constructor call
- `If(cond, then, else)`
- `Let(pattern, value, body)` (pattern may be `ListCons(h, t)` etc.)
- `Lam(arity, [vars], body)`
- Data: `Sym("Nil")`, `Num(n)`, `Bool(b)`, `Var(x)`
- List sugar: `List([...])`, `Cons(head, tail)`

Note. I keep tuple `(k row)` as `List([Var("k"), Var("row")])`, matching how the source uses list.

IL for the chess snippet

```
(= (empty-board) Nil)

(= (adjoin-position $new-row $k $rest-of-queens)
    (append $rest-of-queens (Cons (list ($k $new-row)) Nil)))

(= (select-k $k $positions)
    (if (== $k 1)
        (car-list $positions)
```

```

(select-k (- $k 1) (cdr-list $positions))))

(= (row-safe? $k-col $k-row $positions)
   (null-list? (filter
                 (lambda1 $x (and (== (cdr-list $x) (list ($k-row)))
                                   (not (== (car-list $x) $k-col))))
                 $positions)))

(= (col-safe? $k-row $positions)
   (let $iter (lambda3 $cur $pos $self
                 (if (null-list? $pos)
                     True
                     (if (or (== (cdar-list $pos) (Cons (- $k-row $cur) Nil))
                               (== (cdar-list $pos) (Cons (+ $k-row $cur) Nil)))
                         False
                         ($self (+ $cur 1) (cdr-list $pos) $self))))
     ($iter 1 (cdr-list (reverse $positions)) $iter)))

(= (safe? $k $positions)
    (let (Cons $k-col (Cons $k-row Nil)) (select-k $k $positions)
      (and (col-safe? $k-row $positions) (row-safe? $k-col $k-row $positions))))

(= (queens $board-size)
    (let $queen-cols (lambda2 $k $self
                        (if (== $k 0)
                            (list ((empty-board)))
                            (filter
                             (lambda1 $positions (safe? $k $positions))
                             (flatmap
                              (lambda1 $rest-of-queens
                                    (map (lambda1 $new-row
                                            (adjoin-position $new-row $k $rest-of-queens))
                                          (enumerate-interval 1 $board-size)))
                              ($self (- $k 1) $self)))))
      ($queen-cols $board-size $queen-cols)))

! (assertEqual
   (queens 4)
   (tree (((1 2) (2 4) (3 1) (4 3)) ((1 3) (2 1) (3 4) (4 2)))))
.
```

Key IL translation decisions:

- **Pattern matching:** The `Let` node with `ListCons` patterns preserves MeTTa's unification semantics
- **Closures:** `Lam(arity, vars, body)` directly represents `lambdaN`

- **Control flow:** If nodes mark strict condition, non-strict branches
- **Grounded calls:** All arithmetic/logic remain as **App** nodes for the back-end to implement
- **Data representation:** Position tuples use **List** nodes to match source semantics

7.3.6 Why the snippet behaves correctly under the formal semantics

The implementation's correctness follows from three key properties of our formal semantics:

1. **Evaluation order preservation:** The formal semantics specifies that:

- **if** evaluates condition before selecting branch
- **let** evaluates RHS before pattern matching
- Function arguments evaluate left-to-right (by convention)

These guarantees ensure:

- Recursive calls terminate (base case checked first)
- Pattern destructuring succeeds (values computed before matching)
- Side effects occur predictably (though this code is pure)

2. **Equation-based computation:** Each function is a rewrite rule in the knowledge base:

- Calls trigger Query rule
- Pattern variables unify with actual arguments
- Body instantiation replaces the call
- No special function mechanism needed

This uniform treatment means recursive calls like `(select-k (- k 1) ...)` work identically to top-level calls.

3. **Higher-order programming support:** The semantics treats lambdas as grounded values that:

- Can be bound to variables via **let**
- Can be passed as arguments to **map**, **filter**, etc.
- Can be applied via standard function application
- Support the self-passing pattern for recursion

The combination enables functional abstraction over the search strategy.

7.3.7 A tiny reduction trace (sketch)

To illustrate the complete execution model, we trace key steps of `(queens 4)`:

Initial state:

- Input register I: `{(queens 4)}`
- Knowledge K: All seven function equalities
- Work W: `{}`
- Output O: `{}`

Step 1: Query for queens

- Match `(queens 4)` against `(= (queens $board-size) ...)`
- Unify: `$board-size` $\rightarrow$ 4
- Move instantiated body to W

Step 2: Evaluate let binding

- Create closure for `queen-cols`
- Bind to variable
- Evaluate body: `(queen-cols 4 queen-cols)`

Step 3: Execute queen-cols

- Check `(== 4 0)  $\rightarrow$  False`
- Take else branch
- Recursive call: `(self 3 self)`

Step 4-N: Recursive descent

- Each level generates candidates
- Filters for safety
- Base case `k=0` returns `((Nil))`
- Results propagate back up

Final: assertEqual verification

- Compute `(queens 4)  $\rightarrow$  two solutions`
- Compare with expected result
- Return `()` on success

The trace confirms that complex algorithms emerge naturally from MeTTa's simple rewrite-based evaluation model.

7.4 Concrete MeTTa example using sealed

This example demonstrates MeTTa’s `sealed` construct, which provides hygienic variable renaming to prevent accidental capture. We implement a macro-like function that generates `let` expressions while avoiding variable capture problems that plague naive macro systems.

```
; ----- macro-like expander that produces a let-program -----
(: let-macro (-> Atom Atom Atom Atom)) ; (binder) (rhs) (body) -> expanded program

(= (let-macro $pat $rhs $body)
   (let $body' (sealed ($pat) $body)
     (quote (let $pat $rhs $body')))))

; ----- a concrete call: body mentions both $x and $y -----
! (let-macro $x (+ 1 2) (Cons $x (Cons $y Nil)))
```

The hygiene problem and its solution In macro systems, variable capture occurs when a macro-generated binding accidentally shadows a variable from the calling context. Consider what happens without hygiene:

- The macro generates `(let $x ... body)`
- If the body contains `$y`, and the generated code is later placed inside another `(let $y ...)`, the inner `$y` gets captured
- This breaks referential transparency—the meaning of `$y` changes unexpectedly

MeTTa’s `sealed` solves this by systematically renaming variables:

- `(sealed ($x) body)` renames every variable in body *except* `$x`
- Variables to preserve (here, the binding variable) are listed explicitly
- All other variables get fresh, unique names from a global supply
- Result: `$y` becomes `$y#1`, preventing capture

How it fits the surface syntax The `sealed` special form:

```
(sealed (<Variable> ...) <Atom>)
```

Key syntactic properties:

- First argument: list of variables to preserve (keep unchanged)
- Second argument: the expression to process
- Both arguments have type `Atom` (non-strict positions)

- Non-strictness is crucial: we manipulate the syntax tree, not evaluated values

Interaction with other forms:

The `let` construct:

- Shape: `(let Pattern Form Body)`
- Evaluates `Form`, unifies with `Pattern`, substitutes into `Body`
- `Body` is non-strict (Atom-typed) to enable substitution

The `quote` construct:

- Prevents evaluation of its argument
- Returns the syntax tree as data
- Essential for macro expansion—we generate code, not values

Grammar compliance: All constructs follow the EBNF from Section 2:

- `sealed` matches the special form production with designated non-strict positions
- `let-macro` is a standard definitional equality
- Variable patterns and applications compose normally

Small-step operational semantics walk-through We trace execution through the Meta-MeTTa four-register machine with freshness supply:

$$\langle I, K, W, O \mid \vartheta, M, \nu \rangle$$

Components:

- I : Input register (terms to evaluate)
- K : Knowledge base (equalities and facts)
- W : Workspace (intermediate computations)
- O : Output register (final results)
- ν : Freshness supply (infinite stream of unique names)

Initial configuration:

- K contains: `(= (let-macro $pat $rhs $body) ...)`
- $I = \{(\text{let-macro } \$x \ (+ \ 1 \ 2) \ (\text{Cons } \$x \ (\text{Cons } \$y \ \text{Nil})))\}$
- $\nu = \langle y\#1, y\#2, x\#1, x\#2, \dots \rangle$

Step 1: Query rule application

The machine matches the call against the equality:

- Pattern: `(let-macro $pat $rhs $body)`
- Actual: `(let-macro $x (+ 1 2) (Cons $x (Cons $y Nil)))`
- Unification: $\sigma = \{\$pat \mapsto \$x, \$rhs \mapsto (+ 1 2), \$body \mapsto (Cons \$x (Cons \$y Nil))\}$

Instantiated RHS enters workspace:

```
(let $body' (sealed ($x) (Cons $x (Cons $y Nil)))
  (quote (let $x (+ 1 2) $body')))
```

Step 2: Let evaluation begins

The let rule requires:

1. Evaluate RHS: `(sealed ($x) (Cons $x (Cons $y Nil)))`
2. Unify result with pattern `$body'`
3. Substitute into body

Step 3: Sealed execution (the critical step)

The SEAL rule performs capture-avoiding substitution:

- **Input term:** $t = (Cons \$x (Cons \$y Nil))$
- **Variables in t :** $\{\$x, \$y\}$
- **Variables to preserve:** $\{\$x\}$
- **Variables to rename:** $\overline{V} = \{\$y\}$
- **Fresh name allocation:** Take $y\#1$ from ν
- **Renaming map:** $\rho = \{\$y \mapsto \$y\#1\}$
- **Result:** $\alpha_\rho(t) = (Cons \$x (Cons \$y\#1 Nil))$

State update: $\nu' = \langle y\#2, x\#1, x\#2, \dots \rangle$ (consumed one name)

Step 4: Complete let binding

With sealed result `(Cons $x (Cons $y#1 Nil))`:

- Unify with `$body'`: trivial (variable pattern)
- Binding: $\{\$body' \mapsto (Cons \$x (Cons \$y\#1 Nil))\}$
- Substitute into body:

```
(quote (let $x (+ 1 2) (Cons $x (Cons $y#1 Nil))))
```

Step 5: Output

The quoted term moves to output register O . Final result:

```
(quote (let $x (+ 1 2) (Cons $x (Cons $y#1 Nil))))
```

Note the crucial transformation: $\$y \rightarrow \$y\#1$, while $\$x$ remains unchanged.

Why sealed prevents capture (the key insight) Consider two scenarios:

Without sealed (naive expansion):

```
; Generated code
(let $x (+ 1 2) (Cons $x (Cons $y Nil)))

; If placed in context: (let $y 99 <generated-code>)
; The inner $y refers to the outer binding (99)
; Original meaning lost!
```

With sealed (hygienic expansion):

```
; Generated code
(let $x (+ 1 2) (Cons $x (Cons $y#1 Nil)))

; If placed in context: (let $y 99 <generated-code>)
; The inner $y#1 is distinct from outer $y
; Original meaning preserved!
```

The fresh name `$y#1` cannot conflict with any user-written variable because the freshness supply guarantees uniqueness.

Optional sanity check Direct application shows the renaming clearly:

```
! (sealed ($x) (Cons $x (Cons $y (Cons $y Nil))))
```

Result:

```
(Cons $x (Cons $y#1 (Cons $y#1 Nil)))
```

Both occurrences of `$y` map to the same fresh name, preserving the identity relationship.

Formal specification summary **Syntax:**

```
(sealed (<Variable> ...) <Atom>)
```

Both positions non-strict (marked by parser, preserved by transpiler).

Semantics (SEAL rule):

$$\langle \{(\text{sealed } (v_1 \dots v_k) t)\} \uplus I, K, W, O \mid \nu \rangle \rightarrow \langle \{\alpha_\rho(t)\} \uplus I, K, W, O \mid \nu' \rangle$$

Where:

- ρ : injective renaming for variables not in $\{v_1, \dots, v_k\}$
- α_ρ : capture-avoiding substitution
- ν' : updated freshness supply

Design principle: By evaluating sealed *during* macro expansion (not after), we ensure hygiene before the generated code enters any context where capture could occur.

7.5 Worked example: sequential composition with `progn`

This example demonstrates `progn`, MeTTa’s sequential composition operator. We show how it enables imperative-style programming with side effects while preserving MeTTa’s nondeterministic semantics.

Example program. We create a new space, add one fact to it, print a message, and then return one of two colors nondeterministically.

```
; setup
(bind! &S (new-space))

; helper: print and return its second argument
(: pr (-> Atom $t $t))
(= (pr $msg $v) (progn (println! $msg) $v))

; main: add a fact, print, then choose a color
! (progn
  (add-atom &S (Color Red))
  (pr "after add" 42)
  (superpose ((Color Red) (Color Blue))))
```

Understanding `progn`’s role MeTTa is fundamentally equation-based with nondeterministic evaluation. The `progn` construct provides:

- **Sequential evaluation:** Subterms evaluate left-to-right
- **Side effect ordering:** Effects happen in program order
- **Last-value return:** Result is the value of the final subterm
- **Nondeterminism preservation:** Branches in any subterm are explored at that point

Surface syntax analysis Key syntactic properties:

- **Form:** (`progn` <Form> ...)
- **Strictness:** ALL positions are strict (evaluated)
- **Contrast with `if/let`:** No Atom-typed arguments
- **Grammar production:** Just a regular application where `progn` is the head symbol

Component analysis:

- `add-atom`: Grounded operation with side effect (modifies space)
- `println!`: Grounded I/O operation

- **superpose**: Nondeterminism introducer (creates branches)
- **pr**: User-defined sequencing helper

Operational semantics deep dive The **progn** evaluation rules:

Base: $(\text{progn } e) \rightarrow e$

Step: $(\text{progn } e_1 \ e_2 \ \dots \ e_n) \rightarrow (\text{progn } e_2 \ \dots \ e_n) \text{ after } e_1 \Downarrow v$

Critical properties:

- **Strict left-to-right**: e_1 must fully reduce before considering e_2
- **Effect sequencing**: Side effects of e_1 complete before e_2 starts
- **Error propagation**: If e_i yields Error, entire progn yields Error
- **Empty propagation**: If e_i yields no results, entire progn yields nothing

Detailed reduction trace Initial state: **progn** with three subterms enters input register.

Phase 1: Space modification

- Subterm: `(add-atom &S (Color Red))`
- Symbol `S` already bound to a space via **bind!**
- **E-Add** rule: Adds fact `(Color Red)@S` to knowledge base
- Returns: `()` (unit value)
- State change: `K` now contains the new fact
- Continue with: `(progn (pr "after add" 42) (superpose ...))`

Phase 2: Print with value

- Subterm: `(pr "after add" 42)`
- Query rule matches: `(= (pr $msg $v) (progn (println! $msg) $v))`
- Unification: `$msg` $\rightarrow$ `"after add"`, `$v` $\rightarrow$ `42`
- Expands to: `(progn (println! "after add") 42)`
- Inner progn evaluation:
 1. `(println! "after add")` executes, prints to console, returns `()`
 2. `(progn 42)` reduces to `42`
- Returns: `42`

- Continue with: `(progn (superpose ...))`

Phase 3: Nondeterministic result

- Subterm: `(superpose ((Color Red) (Color Blue)))`
- E_Superpose rule: Creates two branches
- Branch 1: `(Color Red)`
- Branch 2: `(Color Blue)`
- Since this is the last subterm, `progn` returns both values

Observable behavior:

- Console output: `"after add"` (printed exactly once)
- Return values: `{(Color Red), (Color Blue)}`
- Space S: Contains `(Color Red)` fact

The nondeterminism interaction (crucial insight) Consider where non-determinism appears:

- **Last position:** Branches become the `progn`'s results
- **Middle position:** Each branch would continue independently
- **First position:** Side effects could happen multiple times

Example of middle-position branching:

```
(progn
  (superpose (1 2))      ; Creates 2 branches
  (println! "hello"))    ; Prints twice!
```

Our example avoids this by placing nondeterminism last, ensuring single execution of effects.

IL lowering strategy The transpiler converts `progn` to nested `let` with ignore patterns:

$$\mathcal{L}[(\text{progn } e_1 \ e_2 \ e_3)] = (\text{let } _p1 \ \mathcal{L}[e_1] \ (\text{let } _p2 \ \mathcal{L}[e_2] \ \mathcal{L}[e_3]))$$

Why this works:

- `let` evaluates its value argument (enforces sequencing)
- Ignore patterns `_p1`, `_p2` match anything (discard intermediate values)
- Nested structure preserves left-to-right order

- Final expression becomes the result

Concrete lowering:

```
(let _p1 (add-atom &S (Color Red))
  (let _p2 (pr "after add" 42)
    (superpose ((Color Red) (Color Blue))))))
```

Design implications `progn` bridges two paradigms:

- **Functional:** Returns a value, composable, first-class
- **Imperative:** Sequential effects, ordered execution
- **Nondeterministic:** Preserves MeTTa's branching semantics

This enables patterns like:

- Setup-compute-cleanup sequences
- Debugging with print statements
- Transactional operations on spaces
- Imperative algorithms in functional context

7.6 Concrete Examples of User-Configurable Evaluation Order

These examples demonstrate a proposed extension allowing programmers to control evaluation order and strictness per function. This addresses a fundamental tension in language design: different algorithms benefit from different evaluation strategies.

7.6.1 Example 1 — Left-to-right vs right-to-left evaluation

This example shows how evaluation order affects observable behavior when side effects are present.

```
; print-and-return helper (effects only via println!, returns $v)
(: pr (-> String $t $t))
(= (pr $msg $v) (let $u (println! $msg) $v))

; LTR version
(: seq2 (-> $t $t (Pair $t $t)))
(= (seq2 $x $y) (Pair $x $y))
(eval-policy seq2 (args val val) (order ltr))

; RTL version
```

```
(: seq2-rtl (-> $t $t (Pair $t $t)))
(= (seq2-rtl $x $y) (Pair $x $y))
(eval-policy seq2-rtl (args val val) (order rtl))

; Calls to make order observable
! (seq2      (pr "A" 1) (pr "B" 2))    ; prints A then B; result (Pair 1 2)
! (seq2-rtl (pr "A" 1) (pr "B" 2))    ; prints B then A; result (Pair 1 2)
```

The evaluation order problem Most languages fix evaluation order globally:

- **Left-to-right:** C, Java, Python (predictable, matches reading order)
- **Right-to-left:** Some functional languages (enables optimizations)
- **Unspecified:** C++ before C++17 (allows compiler freedom)

MeTTa's approach: Let the function author decide, not the language designer.

Observable behavior For (seq2 (pr "A" 1) (pr "B" 2)) the console prints:

```
A
B
```

For (seq2-rtl (pr "A" 1) (pr "B" 2)) the console prints:

```
B
A
```

Both return (Pair 1 2)—the values are identical, only the effect order differs.

How the policy mechanism works **The policy declaration:**

```
(eval-policy <function> (args <mode>...) (order <spec>))
```

Components:

- **Function:** Which symbol gets the policy
- **Modes:** How each argument is evaluated
 - **val:** Evaluate before call (call-by-value)
 - **name:** Pass unevaluated (call-by-name)
 - **need:** Evaluate on demand with memoization (call-by-need)
- **Order:** In what sequence (ltr, rtl, or custom)

Operational semantics with policies:

When evaluating (f arg1 arg2):

1. Look up policy $\Pi(f)$
2. **Argument preparation phase:**
 - For each **val** argument: evaluate it
 - For each **name** argument: wrap as thunk
 - For each **need** argument: wrap as memoized thunk
 - Follow the specified order
3. Form the call with prepared arguments
4. Apply the function's equation via Query rule

Detailed trace for seq2:

Call: (seq2 (pr "A" 1) (pr "B" 2))

1. Policy lookup: $\Pi(\text{seq2}) = (\text{args: } [\text{val}, \text{val}], \text{order: ltr})$
2. Argument preparation (left-to-right):
 - Evaluate arg1: (pr "A" 1)
 - Prints "A"
 - Returns 1
 - Evaluate arg2: (pr "B" 2)
 - Prints "B"
 - Returns 2
3. Form call: (seq2 1 2)
4. Query matches: (= (seq2 \$x \$y) (Pair \$x \$y))
5. Result: (Pair 1 2)

For **seq2-rtl**, the only difference is step 2 evaluates right-to-left, printing "B" before "A".

Why this matters Evaluation order affects:

- **Side effect sequencing:** I/O, mutations, exceptions
 - **Performance:** Some orders enable better optimization
 - **Resource management:** Order of acquisition/release
 - **Debugging:** Predictable execution aids understanding
- By making it per-function, MeTTa allows:
- Mathematical functions: any order (enables parallelism)
 - I/O functions: strict left-to-right (predictable effects)
 - Control structures: custom patterns (short-circuiting)

7.6.2 Example 2 — Short-circuiting with by-need (lazy) argument

This example implements short-circuit evaluation, where the second argument is only evaluated if necessary.

```
; Short-circuit AND: left strict, right by-need
(: my-and (-> Bool (-> Bool) Bool))      ; suggestive: right treated as thunk
(eval-policy my-and (args val need) (order ltr))

(= (my-and True  $rhs) (force $rhs))      ; force only when left is True
(= (my-and False $rhs) False)

; A right-hand side with a visible effect when forced:
(: rhs-demo (-> Bool))
(= (rhs-demo) (pr "RHS evaluated" True))

! (my-and False (rhs-demo))              ; prints nothing; result False
! (my-and True  (rhs-demo))              ; prints "RHS evaluated"; result True
```

The short-circuiting pattern Short-circuit evaluation is fundamental for:

- **Efficiency:** Avoid unnecessary computation
- **Safety:** Prevent errors (e.g., `x != 0 && 1/x > 2`)
- **Control flow:** Conditional execution without explicit if

Most languages hard-code this for Boolean operators. MeTTa makes it programmable.

Observable behavior analysis Case 1: `(my-and False (rhs-demo))`

- No console output
- Returns `False`
- `rhs-demo` never executes

Case 2: `(my-and True (rhs-demo))`

- Prints "RHS evaluated"
- Returns `True`
- `rhs-demo` executes exactly once

The by-need evaluation model Three evaluation strategies for arguments:

- **By-value (eager):** Evaluate before call
 - Pro: Simple, predictable
 - Con: May do unnecessary work
- **By-name (lazy):** Pass unevaluated, evaluate each use
 - Pro: Avoids unneeded evaluation
 - Con: May duplicate work if used multiple times
- **By-need (lazy with memoization):** Evaluate on first use, cache result
 - Pro: Best of both worlds
 - Con: Requires memoization machinery

Detailed operational semantics Policy specification:

`(eval-policy my-and (args val need) (order ltr))`

Interpretation:

- First argument: `val` (evaluate to Bool)
- Second argument: `need` (wrap as memoized thunk)
- Order: left-to-right (though only first is strict)

Execution trace for `(my-and False (rhs-demo))`:

1. Policy lookup: $\Pi(\text{my-and}) = (\text{val}, \text{need})$
2. Argument preparation:
 - Arg1 (`val`): Evaluate `False` $\rightarrow$ `False`
 - Arg2 (`need`): Create thunk $\theta = \text{thunk}(\text{rhs-demo})$
3. Form call: `(my-and False  $\theta$ )`
4. Query matches: `(= (my-and False $rhs) False)`
5. Unification: $\$rhs \rightarrow \theta$
6. Result: `False`
7. Thunk θ is never forced—no side effects occur

Execution trace for `(my-and True (rhs-demo))`:

1. Same policy lookup and preparation

2. Form call: `(my-and True  $\theta$ )`
3. Query matches: `(= (my-and True $rhs) (force $rhs))`
4. Unification: $\$rhs \rightarrow \theta$
5. Evaluate body: `(force  $\theta$ )`
6. Force operation:
 - Check memo table: not computed yet
 - Evaluate thunk body: `(rhs-demo)`
 - This prints "RHS evaluated" and returns `True`
 - Cache result in memo table
7. Result: `True`

The force/delay mechanism Conceptual rules added to the semantics:
Delay (implicit via policy):

If $\arg_i$ has mode 'need': $e_i \Rightarrow \text{thunk}_\alpha(e_i)$

Force (explicit in equations):

$$\text{force}(\text{thunk}_\alpha(e)) \rightarrow \begin{cases} v & \text{if } \alpha \text{ already computed to } v \\ \text{evaluate } e \text{ and memoize} & \text{otherwise} \end{cases}$$

Design implications This mechanism enables:

- **Custom control structures:** Build your own if/while/for
- **Infinite data structures:** Streams, lazy lists
- **Optimization hints:** Mark expensive arguments as by-need
- **Domain-specific evaluation:** Tailor to algorithm needs

7.6.3 Synthesis: evaluation order as a first-class concern

These examples demonstrate that evaluation strategy profoundly affects:

- **Correctness:** When side effects are present
- **Performance:** What gets computed when
- **Expressiveness:** What patterns are natural

MeTTa's approach-per-function policies-provides:

- **Local reasoning:** Each function documents its strategy

- **Composability:** Different strategies coexist
- **Evolution:** Change strategy without changing callers
- **Optimization:** Choose the best strategy per algorithm

The IL compilation preserves these semantics through:

- Policy tables in module headers
- Explicit delay/force nodes
- Order-respecting argument evaluation
- Thunk representation with memoization support

7.6.4 Takeaway

The key insight: evaluation order is not a language property but a function property. Different functions have different optimal strategies. MeTTa's policy system makes this explicit and programmable, enabling:

- Mathematical functions with parallel evaluation
- I/O functions with strict sequencing
- Control structures with short-circuiting
- Data structures with lazy construction

All coexisting in the same program, each using its optimal strategy.

References

- [1] L. G. Meredith, B. Goertzel, J. Warrell, and A. Vandervorst, *Meta-MeTTa: an operational semantics for MeTTa*, arXiv:2305.17218 (2023).
- [2] B. Goertzel, V. Bogdanov, M. Duncan, D. Duong, Z. Goertzel, J. Horlings, M. Ikle', L. G. Meredith, A. Potapov, A. L. de Senna, H. Seid Andres Suarez, A. Vandervorst, and R. Werko, *OpenCog Hyperon: A Framework for AGI at the Human Level and Beyond*, arXiv:2310.18318 (2023).
- [3] L. G. Meredith, *MeTTa Calculus (draft)*, <https://github.com/leithaus/rho4u/blob/main/metta-calculus/metta-calculus.pdf>
- [4] L. G. Meredith, *MettaCycle Architecture Proposal*, <https://github.com/leithaus/rho4u/blob/main/hypercube/map.pdf>

MeTTa Calculus fragment	Textual MeTTa?IL constructor	IL edge family
<i>Fact in space</i> Σ (persistent): $! \text{emit}(\Sigma, a)$	<code>(add-atom ? a)</code>	E Add (Spaces)
Withdraw one fact a from Σ (consume one guard)	<code>(rem-atom ? a)</code> ⁵	E Remove (Spaces)
Query a space: "match p in Σ then emit r " $\text{for}(p \leftrightarrow \Sigma) \text{emit}(W, r)$	<code>(match ? p r)</code>	E Match (Spaces)
Enumerate matches of p in Σ $\text{for}(p \leftrightarrow \Sigma) \text{emit}(W, p)$	<code>(get-atoms ? p)</code>	E Get (Spaces)
Bidirectional equality rule $\text{for}(t \leftrightarrow \text{Eq}) \text{emit}(W, u) \quad \quad \text{for}(u \leftrightarrow \text{Eq}) \text{emit}(W, t)$	<code>(= t u)</code>	E Query/E Chain (Core)
Directed rule (L?R only) $\text{for}(t \leftrightarrow \text{Eq}) \text{emit}(W, u)$	<code>(=> t u)</code>	E Query/E Chain (L?R)
General "transform" (pattern on facts, produce $u\sigma$) $\text{for}(t \leftrightarrow \Sigma) \text{emit}(W, u)$	<code>(transform t u)</code>	E Transform
Evaluate redex via rules (Query/Chain) $\text{for}(t' \leftrightarrow \text{Eval}) 0 \quad \quad \text{for}(t \leftrightarrow \text{Eq}) \text{emit}(W, u) \rightarrow \text{emit}(W, u\sigma)$	<i>(implicit; no extra constructor)</i>	E Query/E Chain (Core)
Nondeterministic choice (parallel emissions) $\text{emit}(W, a_1) \quad \quad \dots \quad \quad \text{emit}(W, a_n)$	<code>(superpose (a1 ... an))</code>	E Superpose
Materialize all results of e as one tuple (fork?join)	<code>(collapse e)</code>	E Collapse
Alpha?conversion (freshen all vars except listed)	<code>(sealed (\$v1 ... \$vk) t)</code>	E Seal
Fresh space value (private channel/name) $\text{new } \Sigma \text{ in } \{P\}$ (yields a fresh Σ)	<code>(new-space)</code>	Grounder (Spaces)
Grounded operator (host call / arithmetic / I/O) $G(\vec{v})$ emits a ground value	e.g. <code>(+ e1 e2)</code> , <code>(println! a)</code>	Grounders (4.8.6)
Replication (persistent guard) $! \text{for}(t \leftrightarrow x) P$	<code>(= ...)</code> (rules persist) or <code>(add-atom ...)</code> (facts persist)	As above

Table 1: Side-by-side mapping. Calculus fragments use $\text{for}(t \leftrightarrow x)P$, replication $!$, and new with reductions by COMM; see the MeTTa?Calculus note (syntax, COMM, replication, new). The IL column uses the BNFC textual MeTTa?IL constructors and small-step edge families in Sec. 4.7-4.8 of the spec.